FinTrack

A Dissertation submitted in partial fulfillment of the requirement
For the award of degree of

MASTER OF COMPUTER APPLICATIONS

Of

Visvesvaraya Technological University
Belgaum, Karnataka

By

ROHIT T NADAGERI
1CR24MC085

Department of Master of Computer Applications
CMR INSTITUTE OF TECHNOLOGY
132, IT Park Road, Kundalahalli, Bangalore – 560037
2024-2026

FinTrack

A dissertation submitted in partial fulfillment of the requirement
for the award of degree of

MASTER OF COMPUTER APPLICATIONS

Of

Visvesvaraya Technological University

By

ROHIT T NADAGERI
1CR24MC085

Under the Guidance of

Dr. KALAISELVI K
Assistant Professor
CMR Institute of Technology

Department of Master of Computer Applications
CMR INSTITUTE OF TECHNOLOGY
132, IT Park Road, Kundalahalli, Bangalore – 560037

2024-2026

FinTrack

A dissertation submitted in partial fulfillment of the requirement
for the award of degree of

MASTER OF COMPUTER APPLICATIONS

Of

Visvesvaraya Technological University

By

ROHIT T NADAGERI
1CR24MC085

Under the Guidance of

INTERNAL GUIDE

Dr. KALAISELVI K
Assistant Professor
CMR Institute of Technology

EXTERNAL GUIDE

Department of Master of Computer Applications
CMR INSTITUTE OF TECHNOLOGY
132, IT Park Road, Kundalahalli, Bangalore – 560037

2024-2026

CMR INSTITUTE OF TECHNOLOGY

Department of Master of Computer Applications

Bengalore – 560037

CERTIFICATE

This is to certify that the project work entitled

"FinTrack"

Submitted in partial fulfillment of the requirement

For the award of the degree of

Master of Computer Applications of the

Visveswaraya Technological University, Belgaum, Karnataka

Bonafide work carried out by

Rohit T Nadageri

1CR24MC085

Signature of Guide	**Signature of HoD**	**Signature of the Principal**
Dr. Kalaiselvi K	Dr. Gomathi T	Dr. Sanjay Jain
Asst. Professor, MCA	HoD, MCA	Principal, CMRIT

DECLARATION

I, **ROHIT T NADAGERI**, student of the 4th MCA, **CMR Institute of Technology,** Bangalore, bearing USN **1CR24MC085** hereby declare that the project entitled "FinTrack" has been carried out by me under the supervision of Prof. KALAISELVI K, Department of Computer Applications and submitted in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications by the **Visvesvaraya Technological University** during the academic year 2024-25.

This report has not been submitted to any other Organization or University for any award of degree or certificate.

Place: Bangalore Rohit T Nadageri

Date:

1CR24MC085

ACKNOWLEDGEMENT

I would like to extend my heartfelt gratitude to all those who have contributed to the successful completion of this project.

First and foremost, I express my sincere appreciation to my internal project guide, **Prof. Kalaiselvi K**, Assistant Professor in the MCA Department at CMR Institute of Technology, for her continuous encouragement and insightful advice.

I also wish to convey my gratitude to **Dr. Gomathi**, Head of Department and **Dr. Sanjay** Jain, Principal of CMRIT, Bangalore, for there unwavering support during my study.

My deepest appreciation goes to my parents and family members for their unconditional support throughout this project.

Lastly I would like to thank my friends for their constant advice and motivation throughout the course

Rohit T Nadageri
1CR24MC085

Abstract

FinTrac is a personal finance management application developed to assist users in organizing and monitoring their financial activities efficiently. The system provides essential features such as income and expense tracking, budget planning, savings goal management, and multi-account handling within a single platform. The application is designed with a strong focus on the Indian financial environment, supporting Indian Rupee currency formatting, bank details, and UPI-related information to ensure ease of use for local users.

The application enables users to gain a clear understanding of their financial status by presenting structured dashboards, categorized transactions, and visual budget indicators. It allows users to set financial goals, track progress over time, and receive alerts when spending approaches predefined limits. An offline-first approach is adopted so that users can access and manage their financial data without relying on continuous internet connectivity, thereby improving reliability and data security.

The main objective of the project is to demonstrate how modern mobile application technologies can be applied to solve real-world personal finance challenges. FinTrac also highlights the practical implementation of tools such as React Native and local data storage to build a responsive, user-friendly, and secure cross-platform application. Overall, the project serves as an effective example of applying academic knowledge to develop a practical solution that promotes better financial planning and responsible money management.

TABLE OF CONTENTS

Section	Subsection	Page No.
1. Introduction	1.1 Project Description	8
	1.2 Company Description	11
2. Literature Survey	2.1 Existing and Proposed System	12
	2.2 Feasibility Study	14
	2.3 Tools and Technologies Used	15
	2.4 Hardware and Software Requirements	16
3. SRS	3.1 Users	17
	3.2 Functional Requirements	19
	3.3 Non-Functional Requirements	21
4. System Design	4.1 System Perspective	22
	4.2 Context Diagram	24
5. Detailed Design	5.1 Use Case Diagram	26
	5.2 Sequence Diagrams	29
	5.3 Collaboration Diagrams	32
	5.4 Activity Diagram	34
	5.5 Database Design	36
6. Implementation	6.1 Screenshots	39
	6.2 Code Snippets	48
7. Software Testing	Testing & Results	50
8. Conclusion		54
9. Future Enhancements		55
Appendix A	Bibliography	56
Appendix B	User Manual	57

1. Introduction

1.1 Project Description

Managing personal finances has become an essential yet challenging task in today's fast-paced lifestyle. With multiple income sources, daily expenses, digital payments, and long-term financial goals, individuals often struggle to maintain clarity and control over their money. Many existing finance applications are either too complex, require continuous internet access, or do not fully cater to the Indian financial context. To address these challenges, this project titled FinTrac has been developed as a comprehensive personal finance management application.

The primary objective of FinTrac is to provide users with a simple, organized, and reliable platform to track and manage their financial activities efficiently. Instead of using multiple tools or maintaining manual records, users can rely on a single application to record income and expenses, monitor budgets, manage savings goals, and handle multiple accounts. The system is designed with a clear and structured interface that allows users to understand their financial position at a glance.

FinTrac focuses on creating a user-friendly digital environment where financial planning feels manageable rather than overwhelming. Users can add daily transactions by categorizing them into predefined sections such as food, travel, utilities, and others. Budget limits can be set for each category, and visual indicators help users understand how much of the budget has been utilized. This encourages disciplined spending and helps users avoid unnecessary expenses.

The application supports multiple financial accounts, including bank accounts, wallets, and cash records. Users can view balances, track account-wise transactions, and manage financial data in an organized manner. Savings goals are another important feature, allowing users to define targets, set deadlines, and monitor progress over time. These features together help users plan for both short-term needs and long-term financial objectives.

FinTrac is designed using an offline-first approach, ensuring that users can access and update their financial data even without an active internet connection. All user data is stored locally, enhancing privacy and data security. This approach is particularly useful for users who prefer complete control over their financial information without relying on cloud storage.

From a technical perspective, the application is developed using React Native for the frontend to provide a smooth and responsive cross-platform experience. Local storage mechanisms are used to securely store financial records, ensuring fast access and reliable performance. Authentication features such as login and registration ensure that only authorized users can access complete functionality. Once logged in, users are presented with a dashboard that acts as a central hub for navigating transactions, budgets, goals, and reports.

Overall, FinTrac is not just a financial tracking tool but a structured approach to building better money management habits. By combining simplicity, accessibility, and practical financial features, the application helps users gain better awareness of their spending patterns, improve financial discipline, and make informed decisions. The project demonstrates the practical application of academic knowledge to solve a real-world problem and serves as a useful personal finance solution tailored to everyday needs.

2. Literature Survey

2.1 Existing and Proposed System

When it comes to personal finance management, many users already rely on existing mobile and web-based applications such as Mint, ET Money, Walnut, MoneyView, and similar budgeting tools. These applications generally provide basic functionalities like expense tracking, budget creation, bank linking, and financial summaries. Some of them focus more on expense visualization, while others emphasize investment tracking or bill reminders. Although these systems are widely used, they come with several limitations that affect usability and accessibility.

One of the major drawbacks of existing systems is their dependency on continuous internet connectivity. Many applications store user data on cloud servers, making offline access either limited or unavailable. This becomes inconvenient for users who want to record transactions instantly or review their finances without an active internet connection. Additionally, cloud-based storage raises privacy concerns, as sensitive financial data is often shared with third-party services.

Another issue is complexity. Existing finance applications tend to include a wide range of advanced features, charts, analytics, and detailed reports that may confuse average users. While such features may benefit experienced users, they often overwhelm students, beginners, or individuals who simply want to track income and expenses quickly. Some applications also require users to link bank accounts compulsorily, which not everyone is comfortable with.

Cost is another significant limitation. Most existing platforms provide only limited functionality for free and require users to purchase premium subscriptions to access essential features such as detailed reports, multiple accounts, or budget alerts. This makes them less suitable for students and budget-conscious users. Moreover, many applications are not fully optimized for Indian financial practices, such as proper Rupee formatting, Lakh-Crore representation, or simplified UPI-based tracking.

To overcome these limitations, the proposed system FinTrac is designed as a simple, user-friendly personal finance management application. Instead of overwhelming users with unnecessary complexity, FinTrac focuses on providing only the essential features required for effective money management. These include income and expense tracking,

category-wise budgeting, savings goal management, and multi-account handling.

FinTrac is developed with an offline-first approach, allowing users to access and update their financial data without internet dependency. All data is stored locally on the user's device, ensuring better privacy and complete control over personal financial information. The application presents information through a clean dashboard, making it easy for users to understand their financial status at a glance.

Another important advantage of the proposed system is its accessibility. FinTrac does not force users into immediate paid subscriptions and is designed to be student- and beginner-friendly. The interface is straightforward, with clear navigation to transactions, budgets, goals, and reports. Indian currency formatting and localized financial structures further enhance usability for domestic users.

In summary, while existing systems offer useful financial tools, they are often expensive, complex, internet-dependent, and less privacy-focused. The proposed system, FinTrac, aims to provide a simple, secure, and accessible solution that encourages consistent financial tracking and responsible money management without unnecessary technical or financial barriers.

2.2 Feasibility Study

Before developing any project, it is important to analyze whether the proposed system is practical and can be implemented successfully in real-world conditions. This is achieved through a feasibility study, which evaluates the project from different perspectives such as technical, operational, economic, and schedule feasibility. The feasibility study for FinTrac ensures that the system is realistic, cost-effective, and suitable for users.

1. Technical Feasibility

From a technical perspective, the FinTrac project is feasible because it uses reliable, modern, and widely adopted technologies. The frontend of the application is developed using React Native, which allows the system to run smoothly across multiple platforms. Local data storage mechanisms are used to store user financial information securely on the device.

All the tools and frameworks used in the project are open-source, meaning there is no licensing cost involved. These technologies also have strong developer communities, making it easier to find documentation, tutorials, and solutions in case of technical issues. Since the application is designed to work efficiently on standard smartphones and web browsers, it does not require high-end hardware or specialized devices. This makes the technical implementation practical and achievable.

2. Operational Feasibility

Operationally, FinTrac is designed to be simple and user-friendly. The system follows a clear and intuitive workflow that matches real-life financial habits. Users can easily register, log in, and access a personalized dashboard that displays their financial overview. Adding transactions, setting budgets, and tracking savings goals require minimal steps and do not involve complex procedures.

The application is suitable even for users with limited technical knowledge. Most actions, such as recording an expense or checking budget usage, can be completed with a few taps. The offline-first nature of the system also improves usability, as users can manage their finances anytime without worrying about internet availability. Therefore, the system is operationally practical and easy to adopt.

3. Economic Feasibility

From an economic point of view, the FinTrac project is cost-effective to develop and maintain. Since all development tools, frameworks, and libraries used are free and open-source, there is no significant development cost. Local storage reduces dependency on paid cloud services, further minimizing expenses.

The project does not require any additional hardware or external devices, making it affordable for student-level development. If the application is expanded in the future, optional paid services such as cloud backup or advanced hosting can be introduced. However, for the current scope, the overall cost remains minimal, making the project economically feasible.

4. Schedule Feasibility

The project is also feasible in terms of time management. FinTrac is developed in a modular manner, where features such as transaction management, budgeting, account handling, and goal tracking can be implemented independently. This approach ensures that delays in one module do not affect the entire system.

The technologies used are commonly taught and widely practiced, allowing faster learning and implementation. With proper planning and task division, the project can be completed within the allocated academic timeline. Hence, the system is schedule-feasible and manageable within a reasonable development period.

Overall, FinTrac is feasible from all aspects. Technically, it uses stable and accessible technologies; operationally, it is easy to use and practical; economically, it is low-cost; and in terms of scheduling, it can be developed within a realistic timeframe. This makes the project suitable for academic implementation as well as future real-world enhancement.

2.3 Tools and Technologies Used

For developing FinTrac, different tools and technologies were used on both the frontend and backend sides. These technologies were selected because they are free, reliable, and well-supported by the developer community. This helped in smooth development and ensured the application remains stable and easy to maintain.

1. Frontend Technologies

The frontend is the part of the application that users see and interact with directly. In FinTrac, the frontend was built using:

React Native

React Native is a framework used to build cross-platform mobile applications using JavaScript. It allows the same codebase to run on both Android and iOS devices. In FinTrac, React Native was used to develop screens such as the dashboard, transactions page, budget management, savings goals, and account overview.

One of the main reasons for choosing React Native is its component-based structure, which helps in creating reusable UI elements. This improves code organization and reduces development time. React Native also provides a smooth and responsive user experience without reloading screens.

Expo

Expo is a tool built on top of React Native that simplifies development and testing. It was used in FinTrac to speed up the development process and reduce setup complexity. Expo also provides built-in features like icons and development utilities, which helped in creating a clean and consistent interface.

2. Backend Technologies

The backend handles data processing and application logic behind the scenes. Since FinTrac follows an offline-first approach, heavy server-side processing is not required. However, backend logic is handled locally within the application.

JavaScript (Business Logic Layer)

JavaScript is used to implement the core business logic of the application. It manages calculations such as budget usage, balance updates, goal progress tracking, and transaction

handling. Using JavaScript allows better integration between frontend components and

internal logic.

3. Database

AsyncStorage

AsyncStorage is used as the local database for FinTrac. It stores all user-related financial data such as accounts, transactions, budgets, and goals directly on the user's device. This eliminates the need for cloud storage and ensures data availability even without an internet connection.

Using AsyncStorage improves privacy and performance, as sensitive financial data remains under the user's control. It is lightweight, easy to integrate, and well-suited for mobile applications.

4. Authentication and Security

To protect user data, basic authentication features are implemented in FinTrac. Users must register and log in to access their personal dashboard and financial information. Authentication ensures that only authorized users can view or modify stored data.

Sensitive data is handled securely, and access to application features is restricted based on user login status. Since data is stored locally, the risk of data leakage through external servers is minimized, enhancing overall security.

5. Development Tools

Apart from programming frameworks and libraries, several supporting tools were used during development:

Visual Studio Code (VS Code) – Used as the primary code editor. It provides extensions that support React Native development.

Git and GitHub – Used for version control and code backup. These tools help track changes and manage different versions of the project.

npm – Used to manage dependencies and install required libraries.

Jest – Used for basic testing to verify application functionality.

6. Other Libraries and Dependencies

Some additional libraries were used to support specific features in the project:

UUID – Used to generate unique IDs for transactions, accounts, budgets, and goals.

Date-fns – Used for handling dates such as transaction dates, monthly budgets, and goal deadlines.

React Navigation – Used to navigate between different screens in the application smoothly.

The tools and technologies used in FinTrac were chosen mainly for their simplicity, availability, and cost-effectiveness. React Native and Expo helped build a responsive and cross-platform frontend. AsyncStorage handled data storage securely and efficiently. Supporting tools such as GitHub, VS Code, and npm made development more organized. Together, these tools enabled the successful development of FinTrac within the scope of a student-level project.

2.4 Hardware and Software Requirements

Before developing or running any project, it is important to identify the required hardware and software resources. Since FinTrac is a cross-platform personal finance management application designed to run on mobile devices and web browsers, the system requirements are moderate and do not demand high-end equipment. The application can run smoothly on standard devices with basic configurations. The requirements are listed below.

Hardware Requirements

- Processor: Minimum Intel i3 or equivalent
- RAM: 4 GB (8 GB recommended for smoother development and testing)
- Storage: At least 500 MB of free disk space for project files and local data storage
- Display: Standard screen resolution (1366 × 768 or higher)

Internet: Internet connection required during development, testing, and initial setup

The project does not require any additional hardware such as external devices, sensors, or peripherals. This makes the system easy to develop, test, and use on commonly available devices.

Software Requirements

- Operating System: Windows 10 / Linux / macOS
- Frontend Framework: React Native with Expo
- Programming Language: JavaScript / TypeScript

- Local Storage: AsyncStorage
- Code Editor: Visual Studio Code
- Version Control: Git and GitHub
- Testing Tool: Jest (for basic testing)
- Browser (for web testing): Google Chrome / Firefox / Edge (latest versions recommended)

FinTrac does not require expensive hardware or licensed software. A basic laptop or desktop system with an internet connection is sufficient for development and usage. Since all tools and frameworks used are open-source, there is no additional cost involved. This makes the project suitable for student-level development and allows easy scalability and enhancement in
The future.

3. Software Requirements Specification

3.1 Users

Every software system is built for a set of people who are going to use it. In FinTrac, users can be broadly divided into two main categories: guest users and registered users. In addition to these, an admin role can be introduced in the future if required. Each type of user has a different level of access and a different purpose while using the application.

1. Guest Users

Guest users are people who open the application without creating an account. These users are usually first-time visitors who want to understand what the platform offers before signing up. Their access is intentionally limited.

- They can view only sample or demo screens of the application.
- They can explore a basic overview of features such as transaction tracking, budgeting, and goal management.
- They cannot add income or expense transactions.
- They cannot create budgets or savings goals.
- They do not have access to a personal dashboard or data storage.

The main purpose of guest users is to get an idea of how FinTrac works. This allows users to decide whether the application suits their needs before registering.

2. Registered Users

Registered users are those who have created an account and logged into the system. They receive full access to all features of the application.

- They can add, edit, and delete income and expense transactions.
- They can create and manage category-wise budgets and monitor spending limits.
- They can set savings goals and track progress over time.
- They can manage multiple accounts such as bank, wallet, or cash records.
- They can view a personalized dashboard that displays their financial summary.
- All their financial data is saved securely and remains private.

The main purpose of registered users is to use the application regularly as part of their daily financial routine. The dashboard provides a sense of ownership and helps users track their financial activities in an organized manner.

3. Admin Users (optional/future)

At present, FinTrac does not include an admin panel. However, in the future, an admin role can be introduced to manage and maintain the system. Possible responsibilities may include:

- Managing application updates and maintenance.
- Monitoring system performance and stability.
- Handling user feedback and technical issues.

The admin role would not have access to individual user financial data, ensuring privacy and security.

User Flow Summary:

- Guest Users → Limited access for exploration only.
- Registered Users → Full access after login.
- Admin Users (Future) → System management and maintenance.

This user structure is kept simple to ensure ease of use and clarity. Introducing too many user roles could make the system complex and confusing, especially for students and general users. By maintaining clear and limited user roles, FinTrac remains easy to understand, efficient to use, and suitable for everyday personal finance management.

3.2 Functional Requirements

Functional requirements describe what the system must do. These are the core features and actions that FinTrac performs when users interact with it. Without these functions, the application would not fulfill its purpose of helping users manage their personal finances effectively. The main functional requirements of the FinTrac project are listed below.

1. User Authentication

- New users should be able to register by providing basic details such as name, email, and password.
- Existing users should be able to log in using their registered credentials.
- Login credentials must be validated before granting access.
- The system should allow users to log out securely.
- If incorrect login details are entered, the system should display an appropriate error message.

This functionality is necessary to differentiate between guest users and registered users and to protect personal financial data.

2. Dashboard Access

- After successful login, users should be redirected to their personal dashboard.
- The dashboard should display a summary of the user's financial information.
- It should provide navigation options for transactions, budgets, goals, accounts, and reports.
- Guest users should not have access to the dashboard.

3. Account Management

- Registered users should be able to add multiple financial accounts such as bank, wallet, or cash.
- Users should be able to view account balances.
- Users should be able to edit or delete accounts if required.
- The system should update balances automatically when transactions are added.

4. Transaction Management

- Users should be able to add income and expense transactions.
- Each transaction should include amount, category, date, and description.
- Users should be able to view a list of all transactions.
- Users should be able to edit or delete transactions.
- Transactions should be categorized for better tracking.

5. Budget Management

- Users should be able to create monthly budgets for different categories.
- The system should track spending against the set budget.
- Visual indicators should show budget usage.
- The system should alert users when spending approaches or exceeds the budget limit.

6. Savings Goal Management

- - Users should be able to create savings goals with target amounts.
- - Users should be able to set deadlines for goals.
- - The system should track progress towards each goal.
- - Users should be able to add contributions to goals.
- - Goal completion should be clearly indicated.

7. Reports and Analytics

- The system should generate income vs expense summaries.
- Users should be able to view category-wise spending reports.
- Monthly financial trends should be displayed.
- Reports should help users understand their financial behavior.

8. Guest User Flow

- Guest users should be able to explore limited demo features.
- Guest users should not be allowed to add transactions or save data.
- Guest users should see login or signup prompts when attempting to access full features.

9. Error Handling

- The system should display user-friendly error messages for invalid inputs.
- Empty or incorrect fields should not be accepted.
- The application should handle errors gracefully without crashing.

10. Logout and Session Management

- Users should be able to log out at any time.
- After logout, users should not be able to access the dashboard without logging in again.
- User session data should be cleared after logout to ensure security.

The functional requirements of FinTrac focus on providing a complete, simple, and secure personal finance management experience. By supporting transaction tracking, budgeting, goal setting, and reporting, the system helps users manage their finances efficiently while maintaining data privacy and ease of use.

3.3 Non-Functional Requirements

While functional requirements describe what the system does, non-functional requirements explain how well the system should perform. These requirements focus on quality, performance, reliability, security, and user experience rather than specific features. For FinTrac, the main non-functional requirements are described below.

1. Performance Requirements

- The application should load within a few seconds on a normal device.
- Adding or viewing transactions should be fast and responsive.
- Navigation between dashboard screens should not lag.
- Local data operations such as saving budgets or goals should execute smoothly.

- Even with a large number of stored transactions, the system should remain stable.

Since FinTrac is meant for regular financial tracking, smooth and quick performance is important so users can record transactions without delay.

2. Security Requirements

- User login credentials must be handled securely.
- Only registered users should be able to access personal financial data.
- Guest users must not be able to view or modify stored financial information.
- After logging out, users should not be able to access the dashboard without logging in again.
- User data should remain private and protected from unauthorized access.

As FinTrac deals with sensitive financial information, security is a critical requirement of the system.

3. Usability Requirements

- The application should be easy to use without any training.
- The interface should be clean and well-organized.
- Navigation options should be clearly visible on the dashboard.
- Buttons and forms should be simple and understandable.
- The system should be usable by students, professionals, and non-technical users.

Good usability ensures that users can manage their finances comfortably and consistently.

4. Reliability Requirements

- The system should function correctly without frequent crashes.
- User data such as transactions, budgets, and goals should be saved accurately.
- In case of errors, the system should display a proper message instead of failing completely.
- Stored financial data should not be lost during normal usage.

Reliability helps users trust the application and rely on it for long-term financial tracking.

5. Maintainability Requirements

- The codebase should be well-structured and easy to understand.

- Features should be modular so updates can be added easily.
- Bug fixes should not affect existing functionality.

- Future enhancements should be simple to implement.

Since FinTrac is a student project with potential for future development, maintainability is essential.

6. Portability Requirements

- The application should run on Android, iOS, and web platforms.
- It should support major operating systems such as Windows, macOS, and Linux.
- The user interface should be responsive and adapt to different screen sizes.

Portability ensures that the system is accessible to users across multiple devices.

7. Scalability Requirements

- The system should support an increasing number of transactions per user.
- Storage should handle growing financial records over time.
- The application should allow future upgrades if usage increases.
- Additional features can be added without redesigning the entire system.

Scalability ensures that FinTrac can grow beyond a small student project and support more users if needed.

The non-functional requirements of FinTrac ensure that the system is not only functional but also reliable, secure, user-friendly, and future-ready. These qualities are important because even a feature-rich system will fail if it is slow, unsafe, or difficult to use. Together, these requirements help make FinTrac a trustworthy and practical personal finance management application.

4. System Design

System design forms the backbone of the project, as it explains how different components of the system work together to deliver the required functionality. While the requirement specification phase focused on what the system should do, the design phase focuses on how the system achieves those requirements in a structured and efficient manner.

The design of FinTrac has been planned in such a way that the system remains modular, scalable, and easy to maintain. Each major module such as transaction management, budget tracking, savings goals, and account management operates independently but interacts through a common application structure. This modular approach allows individual features to be updated or extended without affecting the entire system.

At a high level, FinTrac follows a three-tier architecture, which separates the system into distinct layers. This separation improves clarity, maintainability, and overall system performance.

Presentation Layer (Frontend)

The presentation layer is responsible for providing the user interface through which users interact with the application. In FinTrac, this layer is developed using React Native and Expo. It includes screens such as login and registration, dashboard, transactions, budgets, goals, and account management. This layer handles user input, displays financial information, and provides navigation between different sections of the application.

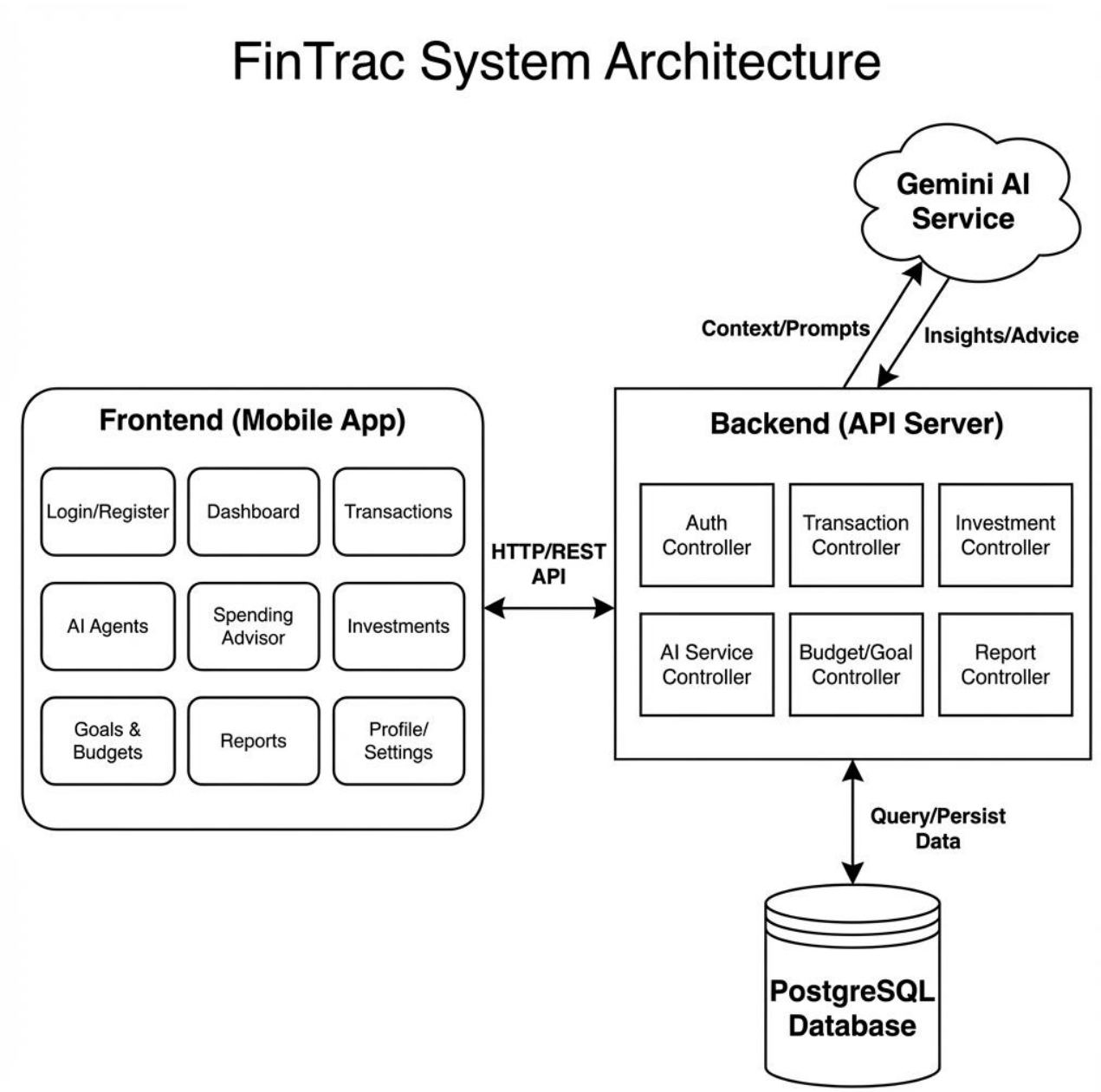
Application Layer (Business Logic)

The application layer contains the core logic of the system. It processes user actions such as adding transactions, updating budgets, calculating balances, and tracking goal progress. This layer ensures that data is validated before being stored and that business rules are followed consistently. It acts as a bridge between the user interface and the data storage layer.

Data Layer (Local Storage)

The data layer is responsible for storing and retrieving user data. In FinTrac, this is handled using local storage mechanisms such as AsyncStorage. Financial data including accounts, transactions, budgets, and goals are stored securely on the user’s device. This approach supports offline usage and enhances data privacy, as information is not dependent on cloud servers.

The following diagram illustrates the high-level architecture of the system:



1.1 System Perspective

The system design of FinTrac is built around a simple and well-structured architecture. It follows a modular application model where the frontend interacts with the internal application logic and local data storage to manage financial information efficiently. Since FinTrac is designed as an offline-first application, most operations are performed locally on the user's device, ensuring privacy and availability without continuous internet dependency.

At a high level, the system consists of three main components:

1. **Frontend (User Interface):**
    - Developed using React Native with Expo.
    - This layer is what the user directly sees and interacts with.
    - It includes screens such as login/signup, dashboard, transactions, budgets, savings goals, accounts, and reports.
    - The frontend captures user input and displays financial data in a clear and organized format.
    - It communicates with the application logic layer to process user actions.
2. **Backend (Server Logic):**
    - Implemented using JavaScript and React Native logic.
    - This layer handles core functionalities such as transaction validation, budget calculations, balance updates, and goal progress tracking.
    - It manages user authentication flow and ensures that only authorized users can access personal financial data.
    - It acts as a bridge between the user interface and the data storage layer.
3. **Database (Storage Layer):**
    - Uses AsyncStorage for local data persistence.
    - Stores user-related information such as accounts, transactions, budgets, and goals.
    - Data is saved in structured key-value format.
    - Each user's data is stored separately to avoid data mixing and ensure privacy.

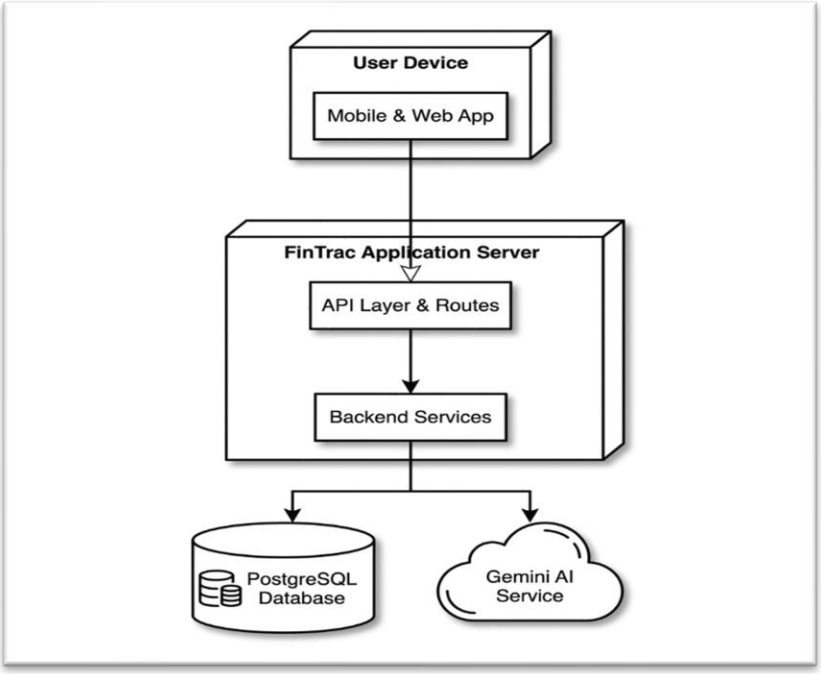
System Integration

- The user performs an action on the frontend, such as adding a transaction or creating a budget.
- The application layer validates the input and applies business rules.
- The data layer stores or retrieves the required information.
- The processed data is then sent back to the frontend and displayed to the user.

External Systems and Constraints

- Since FinTrac stores data locally, internet access is not required for daily usage.
- Internet connectivity is only needed during development, updates, or future enhancements.
- The application depends on devices supporting modern mobile operating systems and JavaScript.
- Storage limitations of the device may affect the amount of data stored.

The following diagram illustrates the system perspective of Fintrac in relation to its components and external environment.



In simple words, FinTrac follows a three-layer structure consisting of the frontend, application logic, and data storage layers. Each layer has a specific responsibility, but they work together to provide users with a smooth and reliable experience. This system perspective shows that

FinTrac is lightweight, well-organized, and easy to maintain, making it suitable for future Enhancements and expansion

1.2 Context Diagram

A context diagram represents a high-level view of the system and shows how it interacts with external entities. It is also referred to as a Level 0 Data Flow Diagram (DFD). The main purpose of this diagram is to represent the entire system as a single process and illustrate its interaction with users and other components.

For the FinTrac system, the main external entities are:

1. Guest User
2. Registered User
3. Admin (Future Scope)
4. Local Storage (AsyncStorage)

The main system is represented as the FinTrac Application.

How the Interactions Work:

Guest User → FinTrac:

- Can explore limited demo features of the application.
- Can view sample dashboards or feature previews.
- Cannot add transactions, budgets, or goals.
- No personal financial data is stored for guest users.

Registered User → FinTrac:

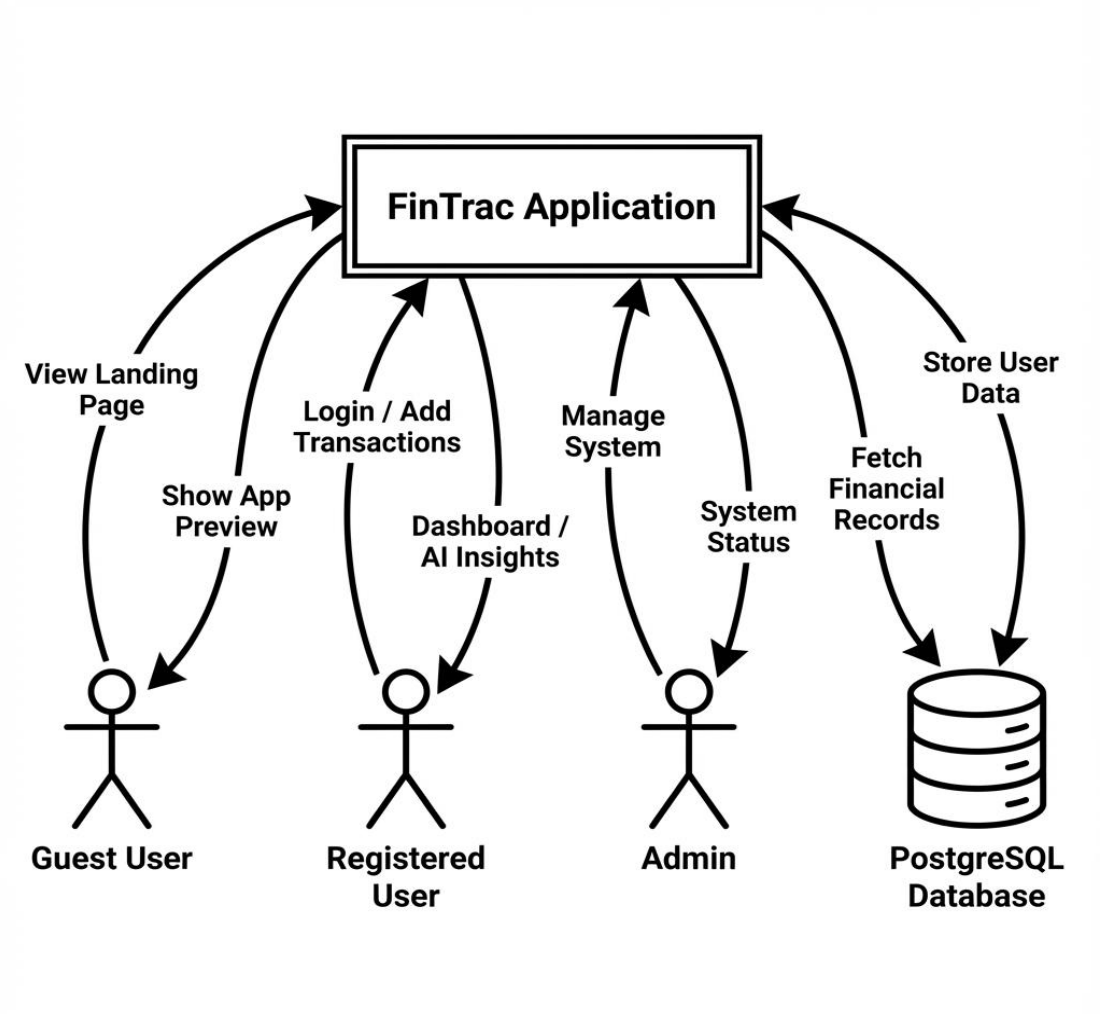
- Logs in using valid credentials.
- Can add, edit, and view income and expense transactions.
- Can create and manage budgets and savings goals.
- Can manage multiple accounts such as bank, wallet, or cash.
- Receives a personalized dashboard displaying financial summaries.

Admin (Future Role) → FinTrac:

- Manages application maintenance and updates.
- Handles system-level configurations.
- Monitors application performance and stability.
- Does not access individual user financial data.

FinTrac → Local Storage (AsyncStorage):

- Stores user accounts, transactions, budgets, and goals.
- Retrieves stored financial data for display on the dashboard.
- Updates stored data when users modify or add records.
- Maintains data separation for different users.



The context diagram of FinTrac explains the interaction between users, the application, and the data storage layer. It shows that guest users are given limited access for exploration, registered users have full access to manage their financial data, and a possible admin role handles system maintenance in the future. All user data such as transactions, budgets, and goals is managed through local storage. This diagram is useful because it provides a clear high-level view of the system before moving into more detailed design stages.

5. Detailed Design

5.1 Use Case Diagram

A Use Case Diagram is a type of UML diagram that explains how different users interact with the system. It focuses on the system's functionality from the user's point of view rather than technical implementation. For the FinTrac project, three types of users are considered: Guest User, Registered User, and Admin (future role). Each user has different responsibilities and access levels.

Actors in the System

1. Guest User

A guest user is anyone who opens the FinTrac application without logging in. The main purpose of a guest user is to explore the application before registering. Guest users are given limited access so that they understand the value of the system while still being encouraged to create an account. Guest users can:

- View sample or demo screens of the application.
- Explore basic features such as transaction tracking and budgeting in preview mode.
- Navigate through the application interface.
- Sign up or log in to access full functionality.

These limitations are intentional so that users register to store and manage their personal financial data securely.

2. Registered User

Registered users are the primary users of the FinTrac system. Once a user signs up and logs in, they gain full access to all features of the application. Registered users can:

- Add, edit, and delete income and expense transactions.
- Create and manage monthly budgets.
- Set savings goals and track their progress.
- Manage multiple financial accounts such as bank, wallet, or cash.
- View a personalized dashboard displaying financial summaries.
- View reports and financial trends.
- Log out securely from the application.

Registered users make the application meaningful because their financial data is stored securely

and can be accessed whenever they log in.

3. Admin (Future Role)

Currently, FinTrac is designed mainly for end users. However, in the future, an Admin role can be introduced to manage and maintain the system. The admin would be responsible for:

- Managing application updates and maintenance.
- Monitoring system performance.
- Handling user-related issues or misuse.
- Ensuring the smooth functioning of the application.

Although the admin role is not part of the current implementation, it will be important if the system is expanded for large-scale usage.

Use Cases in FinTrac

Below is the detailed explanation of each use case:

Use Case	Actor	Description	Goal
Login	Registered User	User enters email and password. The System checks Credentials in PostgreSQL. If valid, they get access to dashboard.	Authenticate user
Sign Up	Guest User	Guest provides details (name, email, password). The system creates a new account.	Register new user
View Dashboard	Registered User	User views financial summary, charts, and recent transactions.	Overview of finances
Add Transaction	Registered User	User inputs income or expense details (amount, category, date). Data is stored.	Record financial activity

AI Insights	Registered User	User asks questions or views spending advice from Gemini AI.	Receive financial guidance
Manage Budgets	Registered User	User sets limits for categories. System tracks progress and alerts on overspending.	Control spending
View Reports	Registered User	User views detailed analysis and generates PDF reports of financial health.	Analyze financial trends
Manage Profile	Registered User	User updates personal info, currency settings, or preferences.	Maintain account details
Admin Panel	Admin	Admin manages users and system configurations (Future Scope).	System administration

Login

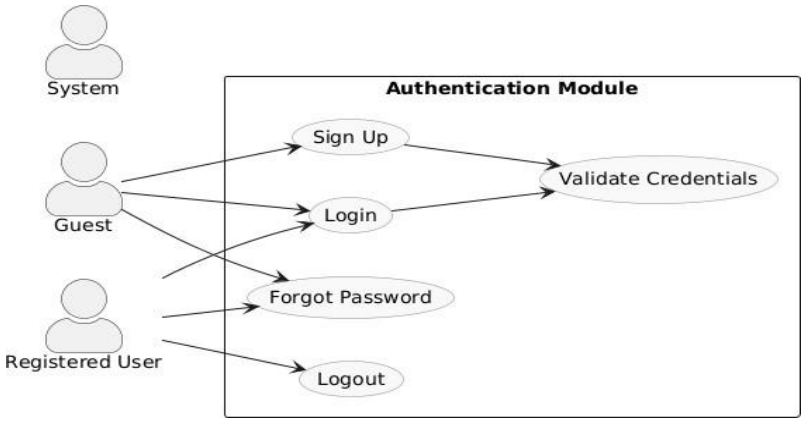
The login use case allows registered users to access the FinTrac application securely. In this process, the user enters their registered email and password. The system validates these credentials and verifies the user's identity. If the entered details are correct, the user is successfully logged in and redirected to the personal dashboard. If the credentials are incorrect, the system displays an appropriate error message and denies access.

This use case ensures that only authorized users can access personal financial data such as transactions, budgets, and savings goals. It plays a crucial role in maintaining data privacy and security within the system.

Sign Up

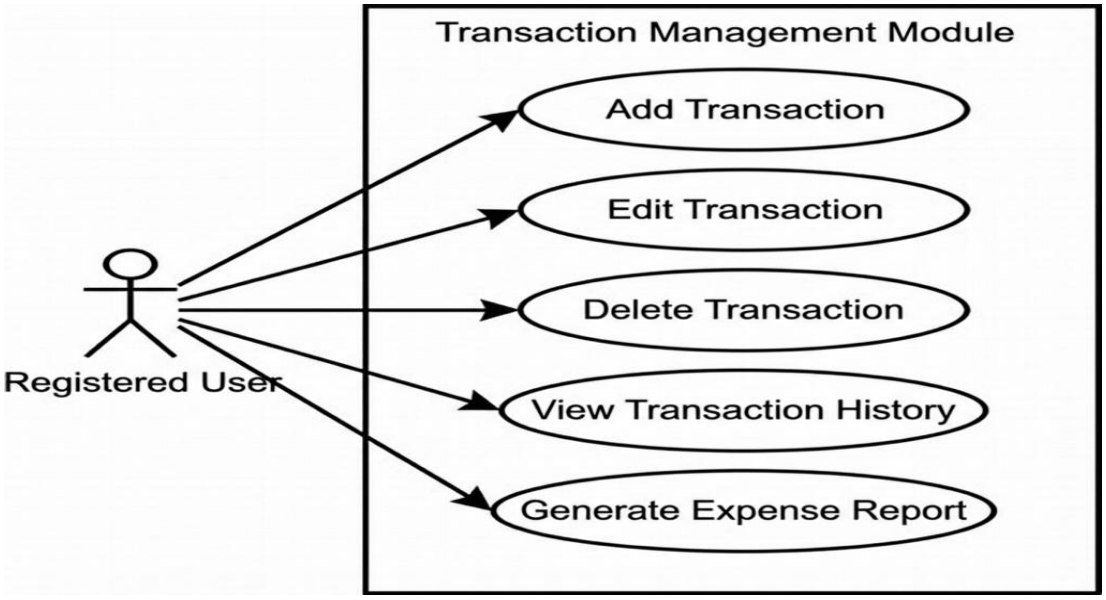
The sign-up use case allows new users to create an account in the FinTrac system. Guest users who want to access full features such as transaction tracking, budget management, and goal setting must register first. During registration, users are required to provide basic details such as name, email address, and password.

Once the details are submitted, the system creates a new user account and stores the information securely. After successful registration, users can log in anytime using their credentials. This use case is important because it converts first-time visitors into registered users and enables long-term use of the application for managing personal finances.



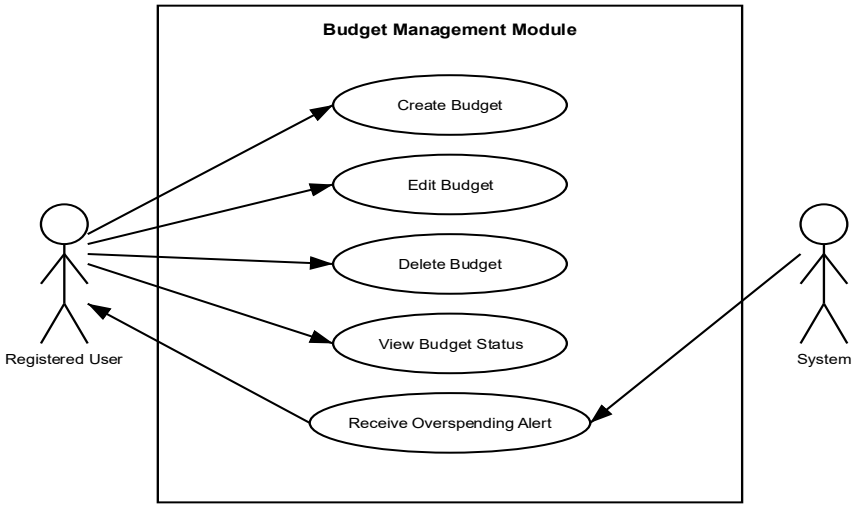
Transaction Management

Registered users can use this use case to record their financial activities. A user can add a transaction by selecting whether it is an income or an expense, entering the amount, choosing a category, and providing an optional description. Each transaction is automatically saved with the date and time. Users can later view the history of all transactions, edit existing entries, or delete them if needed. This use case helps users keep a clear and organized record of their daily spending and earnings.



Budget Management

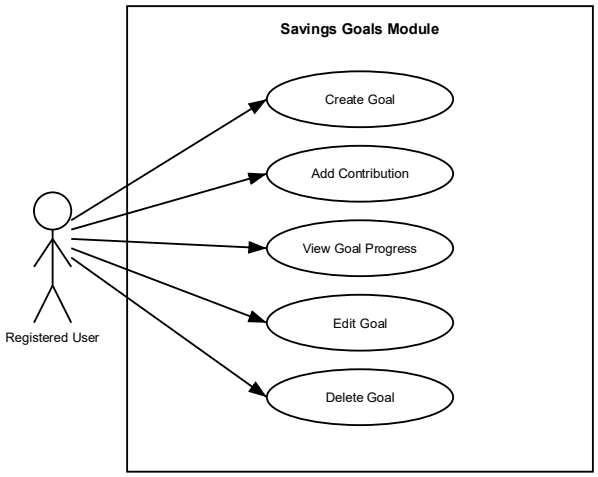
The budget management use case allows users to plan their monthly expenses. Users can set a budget limit for different categories such as food, travel, or utilities. Once a budget is created, the system continuously tracks spending against the defined limit. Users can view how much of the budget is used and how much remains. Budgets can be modified or removed if financial plans change. This feature supports financial discipline and helps users avoid overspending.



Savings Goals

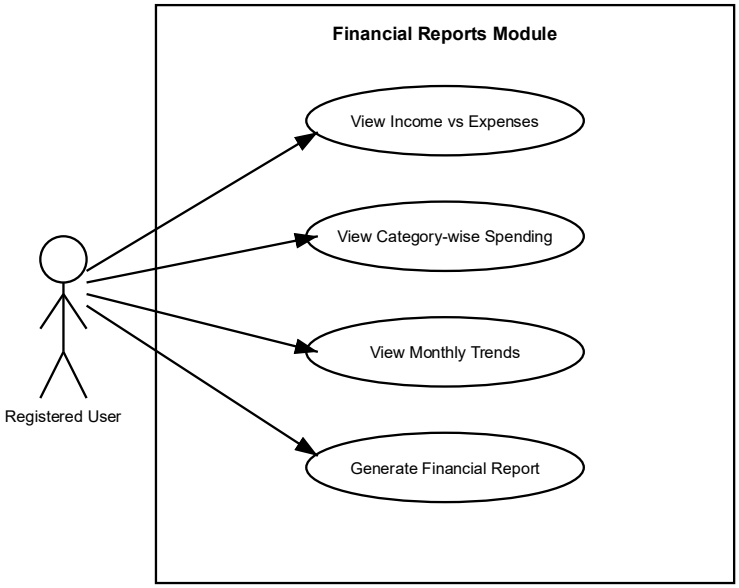
Savings goals are accessible only to registered users. In this use case, users can create a goal by specifying a target amount and a deadline. Contributions can be added over time, and the system automatically updates the progress. Users can view goal completion percentage and

track how close they are to achieving their target. Goals can also be edited or deleted if required. This use case encourages long-term financial planning and motivation.

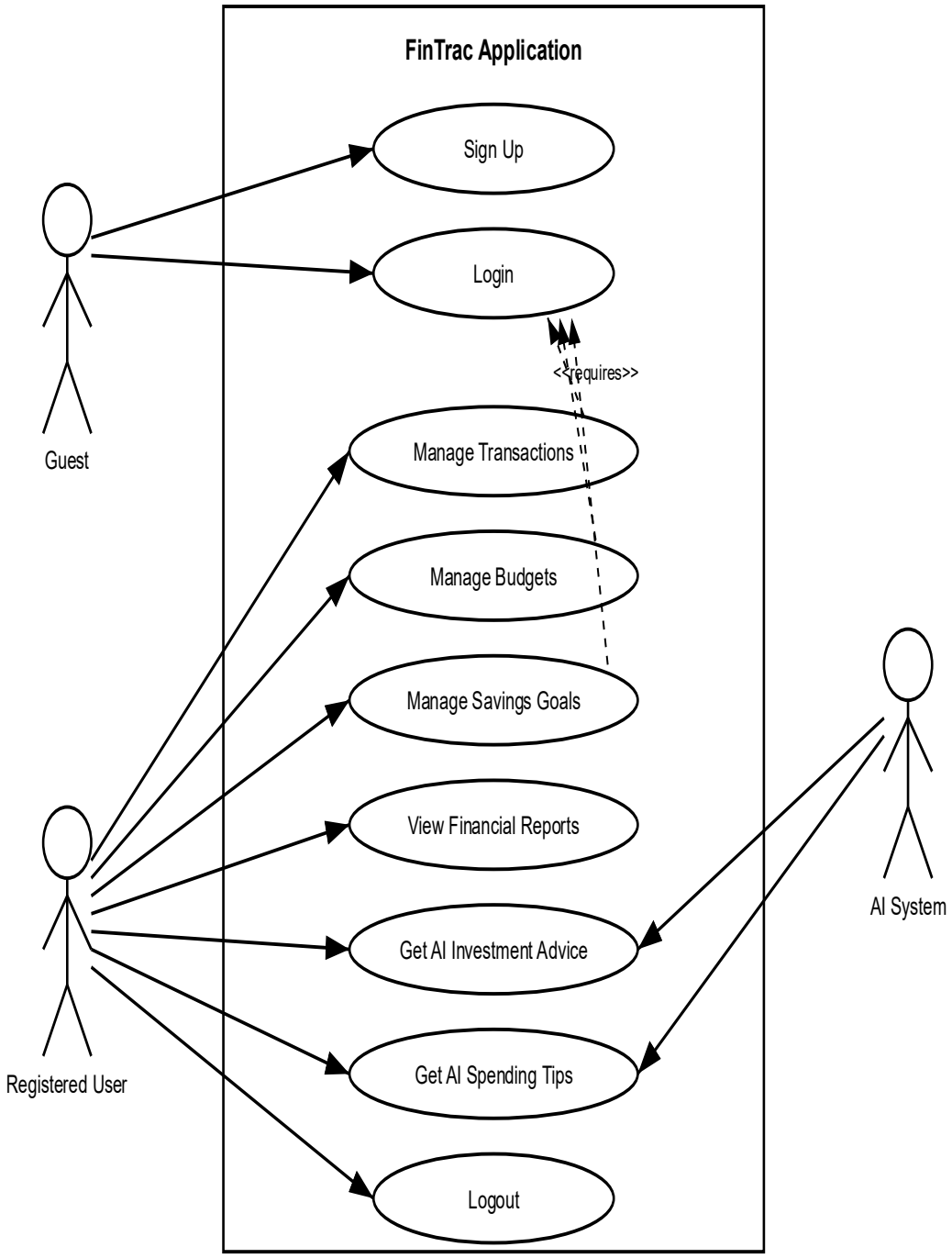


Financial Reports

The reports use case allows users to analyze their financial behavior. Users can view summaries such as income versus expenses, category-wise spending, and monthly trends. These reports are generated from stored transaction data and presented in a simple and understandable format. Reports help users reflect on their financial habits and make informed decisions for the future.



The use case diagram for FinTrac covers all the key interactions within the system. Guest users are allowed limited access to explore the application, but they must register to use the full set of features. Registered users have complete access to manage transactions, budgets, goals, and reports, while a future admin role helps in maintaining and managing the system. By clearly separating user roles and access levels, FinTrac ensures both security and ease of use.



5.2 Sequence Diagrams

A sequence diagram illustrates the order in which actions occur within a system. It is a type of UML diagram that focuses on time-based interactions between the user, the application interface, and the data storage layer.

The diagram uses lifelines (vertical dashed lines) and messages (horizontal arrows) to represent the step-by-step flow of operations.

In FinTrac, sequence diagrams help explain how user requests such as login, adding transactions, or saving budgets move through the system and interact with local data storage.

Sequence Diagram for Login Steps

1. The user opens the FinTrac login screen.
2. The user enters their registered email and password.
3. The system sends the login request to the authentication logic.
4. The authentication module checks the entered credentials against the stored user data.
5. The data storage layer responds with a validation result (valid or invalid).
6. If the credentials are valid, the user is redirected to the personal dashboard.
7. If the credentials are invalid, an error message is displayed on the screen

1. Sequence Diagram for Sign Up

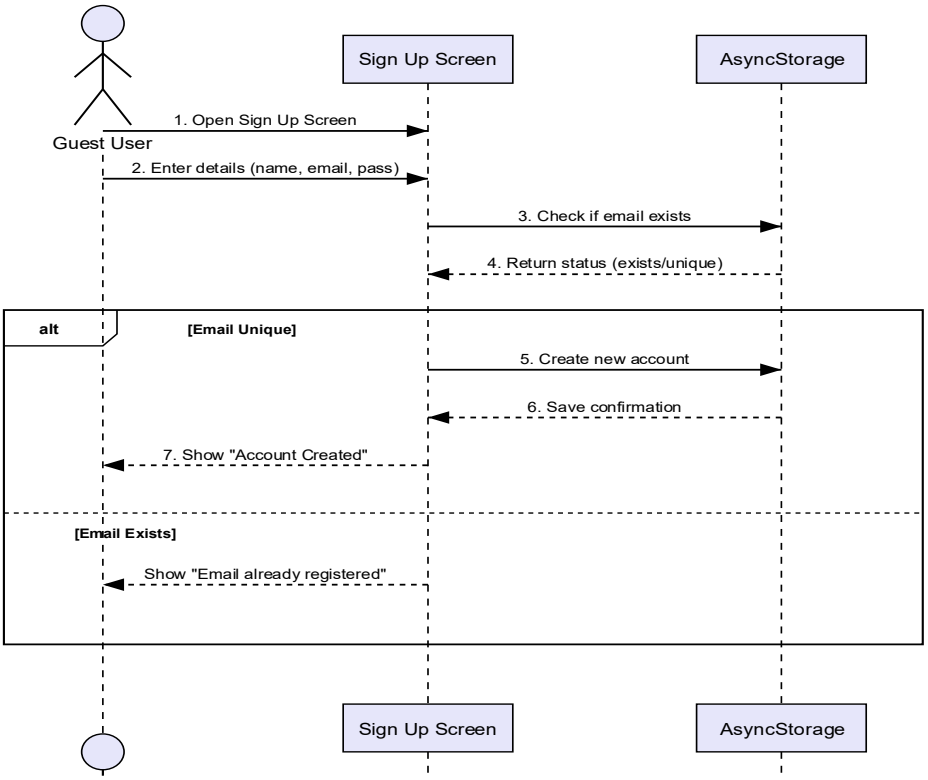
Steps

1. Guest opens the sign-up page.
2. Guest enters details (name, email, password).
3. System checks if email already exists in MongoDB.
4. If unique, the system creates a new account.
5. Confirmation message is shown to the user.

Lifelines and Messages

Actor	Message	Receiver	Description
User	Enter email & password	Login Screen	Provides login credentials
Login Screen	Send login request	Authentication Module	Forwards login request
Authentication Module	Verify credentials	Local Storage	Checks stored user data
Local Storage	Return result	Authentication Module	Valid or Invalid response
Authentication Module	Display output	User	Dashboard access or error message

This sequence prevents unauthorized users from accessing the FinTrac system.



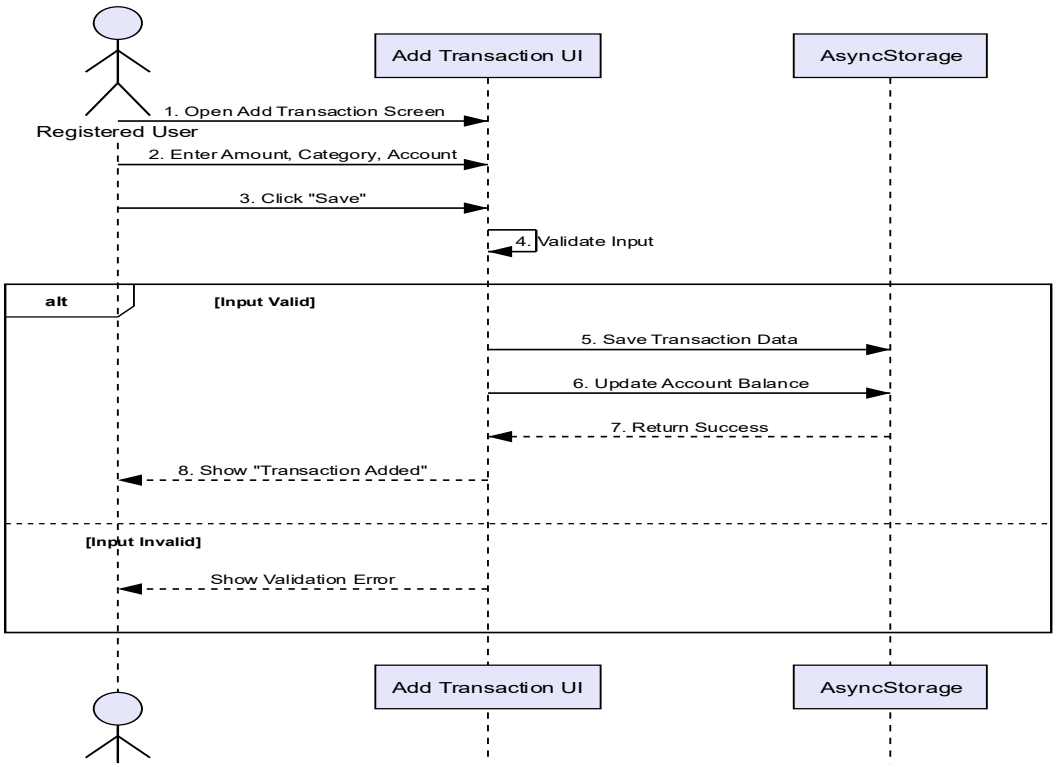
2. Sequence Diagram for Transaction Management Steps

- Registered user clicks on the Add Transaction option.
- The system displays the transaction form with fields such as amount, category, type (income/expense), and date.
- User enters the transaction details and submits the form.
- The system validates the entered data and stores the transaction in local storage along with the date and user ID.
- Confirmation is displayed on the screen, and the updated balance is reflected on the dashboard.

Lifelines and Messages

Actor	Message	Receiver	Description
User	Enter transaction details	Transaction Page	Provides transaction data
Transaction Page	Send data	Transaction Controller	Processes user input
Transaction Controller	Save transaction entry	Local Storage	Stores transaction with date and user ID
Local Storage	Respond success	Transaction Controller	Confirms transaction saved
Transaction Controller	Display confirmation	User	Shows 'Transaction saved'

This sequence ensures that all transactions entered by the user are securely stored and properly reflected in the system.



2. Sequence Diagram for Budget Management

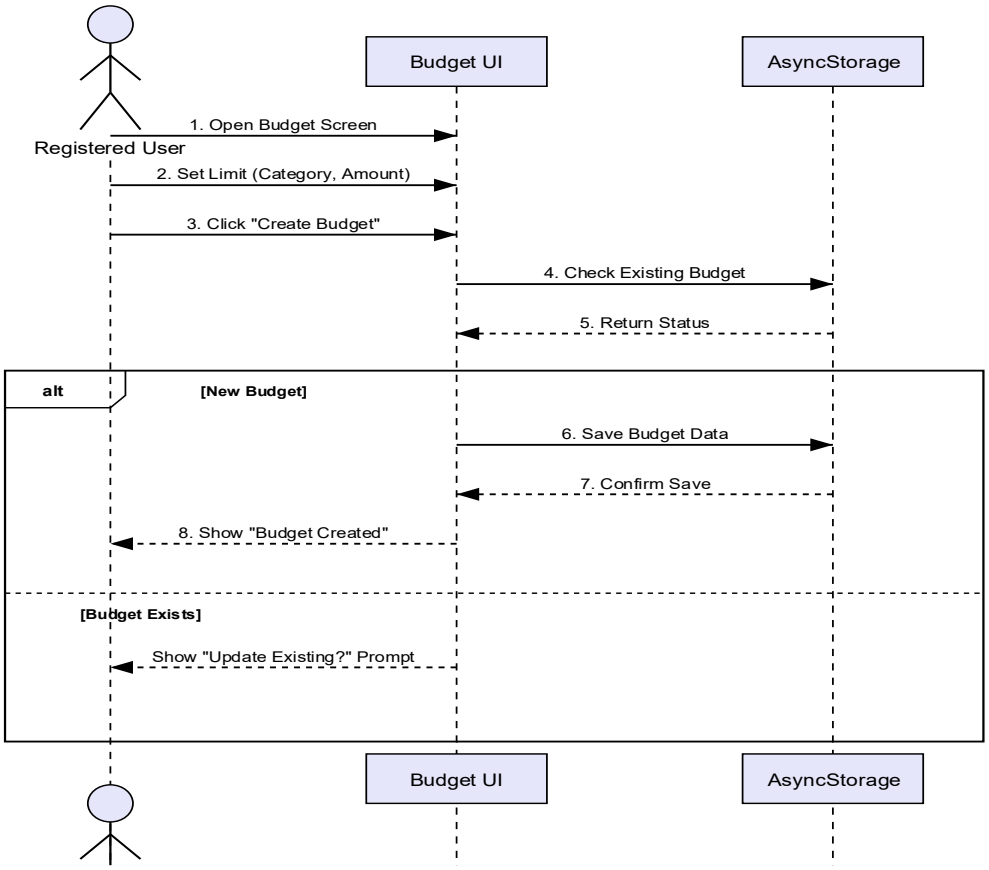
Steps

1. Registered user opens the Budget Management page.
2. User enters budget details such as category and monthly limit.
3. The system sends the budget data to the budget controller.
4. The budget controller stores the data in local storage linked to the user ID.
5. A confirmation message is displayed to the user.
6. When the user views budgets later, stored budget data is retrieved and displayed.

Lifelines and Messages

Actor	Message	Receiver	Description
User	Enter budget details	Budget Page	Provides category and limit
Budget Page	Send budget data	Budget Controller	Processes request
Budget Controller	Save budget	Local Storage	Stores budget with user ID

Local Storage	Confirm save	Budget Controller	Budget saved successfully
Budget Controller	Display confirmation	User	Shows 'Budget saved'
User	Request budget view	Budget Controller	Wants budget history
Budget Controller	Fetch budgets	Local Storage	Retrieves stored budgets
Local Storage	Send budgets	Budget Controller	Returns saved budgets
Budget Controller	Show budgets	User	Displays budget list



Sequence Diagram for Savings Goal

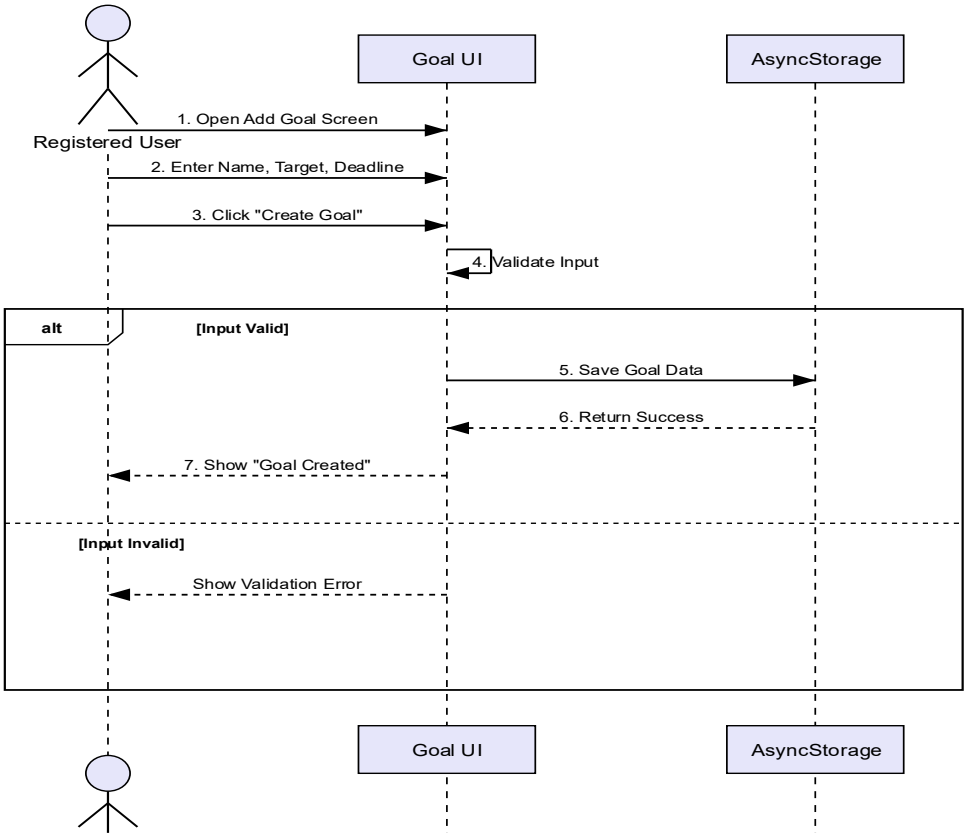
Steps

1. User opens the Savings Goals page.
2. The Goals Page requests saved goals from the Goals Controller.
3. The Goals Controller fetches goal data from local storage.
4. Stored goals are returned and sent back to the Goals Page.
5. The Goals Page displays the list of savings goals to the user.
6. User creates or updates a savings goal by entering target amount and deadline.
7. The system stores the goal details linked to the user.
8. Confirmation is displayed and goal progress is updated.

Lifelines and Messages

Actor	Message	Receiver	Description
User	Open Goals Page	Goals Page	Starts the request
Goals Page	Request goals	Goals Controller	Sends request for goal data
Goals Controller	Fetch goals	Local Storage	Retrieves goals based on user ID
Local Storage	Return goals	Goals Controller	Responds with saved goals
Goals Controller	Send goals	Goals Page	Provides goal list
Goals Page	Display goals	User	Shows goals on screen
User	Add / update goal	Goals Page	Submits goal details
Goals Page	Save goal	Goals Controller	Processes goal request
Goals Controller	Store goal	Local Storage	Saves goal with progress
Local Storage	Confirm save	Goals Controller	Goal saved successfully
Goals Controller	Display confirmation	User	Shows goal saved message

This flow ensures that guest users can access only limited preview features, while registered users are granted full access to manage their financial data. It also allows users to view detailed information such as individual transactions, budgets, or savings goals by retrieving the complete data stored in the system.



The sequence diagrams of FinTrac illustrate how the system processes key use cases in a step-by-step manner. Login and Sign Up operations ensure secure access to the application, while features such as transaction management, budget planning, and savings goal tracking help users manage their finances effectively. Each sequence demonstrates proper interaction between the user, the application interface, and the data storage layer, ensuring accuracy, security, and smooth system operation.

Collaboration Diagrams

A Collaboration Diagram is another UML design diagram that shows how objects connect with one another. It resembles a sequence diagram, but instead of showing vertical timelines, it emphasizes the relationships (links) between objects.

In short:

- Sequence Diagram → focuses on “when” interactions happen.

- Collaboration Diagram → focuses on “who talks to whom” in the system.

Both diagrams are complementary, and in the FinTrac project, they help in understanding how the system is structured and how different components interact with each other in a clear and organized manner.

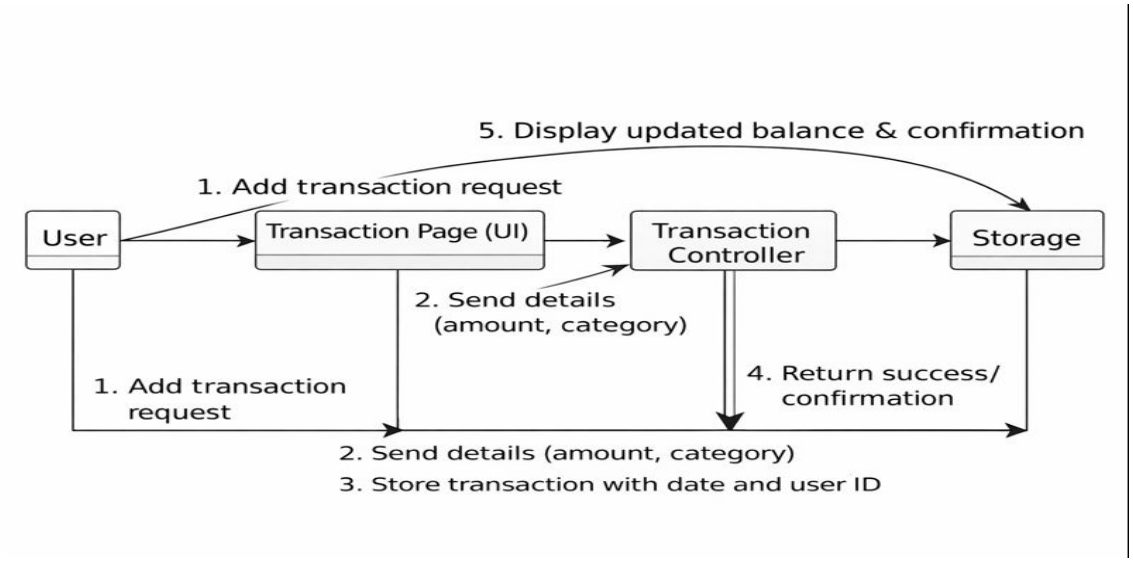
1. Collaboration Diagram for Transaction Management

Objects Involved

1. User
2. Transaction Page (User Interface)
3. Transaction Controller
4. Local Storage (AsyncStorage)

Interactions

1. The user enters transaction details on the Transaction Page.
2. The Transaction Page forwards the transaction data to the Transaction Controller.
3. The Transaction Controller validates the transaction details.
4. The Transaction Controller stores the transaction data in Local Storage linked with the user ID.
5. Local Storage returns a confirmation message to the Transaction Controller.
6. The Transaction Controller sends a success or failure response to the Transaction Page.
7. The Transaction Page displays a confirmation message or error to the user.



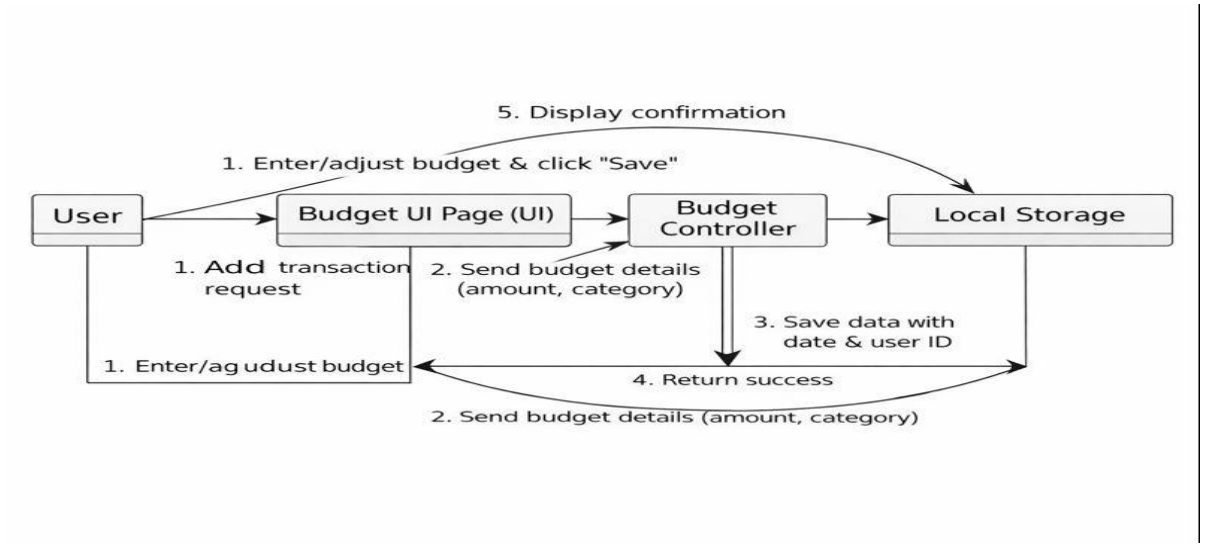
2. Collaboration Diagram for Budget Management

Objects Involved

- User
- Budget Management UI Page
- Budget Controller
- Local Storage (AsyncStorage)

Interactions

1. User enters budget details such as category and limit and clicks the 'Save' button.
2. The Budget Management UI forwards the request to the Budget Controller.
3. The Budget Controller validates the budget data and stores it in Local Storage.
4. Local Storage confirms successful saving of the budget.
5. The Budget Controller notifies the UI, which displays a confirmation message to the user.
6. When the user requests to view budgets, the Budget Controller retrieves budget data from Local Storage and displays it on the UI.



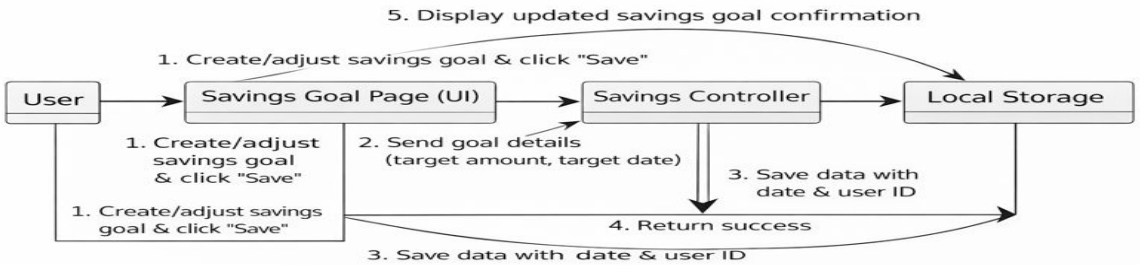
3. Collaboration Diagram for Saving Goal Management

Objects Involved

- User
- Savings Goal Page (UI)
- Savings Controller
- Local Storage (AsyncStorage)

Interactions

1. The user opens the Savings Goal page and enters goal details such as target amount and target date.
2. The Savings Goal Page sends the entered details to the Savings Controller.
3. The Savings Controller validates the data and prepares it for storage.
4. The Savings Controller stores the savings goal information in Local Storage along with the user ID.
5. Local Storage sends a success confirmation back to the Savings Controller.
6. The Savings Controller notifies the Savings Goal Page about the successful operation.
7. The Savings Goal Page displays a confirmation message and updated goal progress to the user.
8. When the user revisits the goals page, stored savings goals are retrieved and displayed.



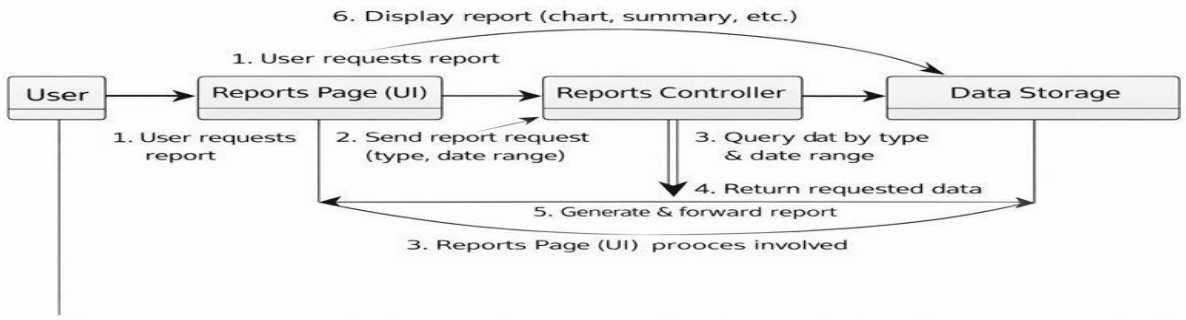
1. **Collaboration Diagram for Financial Reports**

Objects Involved

1. User
2. Financial Reports Page
3. Reports Controller
4. Local Storage (AsyncStorage)

Interactions

1. User opens the Financial Reports page.
2. Financial Reports Page sends a request to the Reports Controller.
3. Reports Controller queries Local Storage for transaction and budget data.
4. Local Storage returns the required financial data.
5. Reports Controller processes the data and generates summaries.
6. Reports Controller forwards the report data to the Financial Reports Page.
7. Financial Reports Page displays charts and summaries to the user.
8. If the user changes filters (date/category), the process repeats from Step 2.



The collaboration diagrams of FinTrac (Login, Transaction Management, Budget Management, Savings Goals, and Financial Reports) clearly show how different system components interact with each other. They help in understanding how various modules work together to complete a single operation in an organized manner.

- Transaction Management collaboration shows how users add, store, and retrieve income and expense records through proper coordination between the user interface, controller, and storage layer.

- Budget Management collaboration illustrates how users create and manage budgets, and how budget data is saved and fetched to track spending limits.

- Savings Goals collaboration demonstrates how users set financial goals, update progress, and view goal details in a structured flow.

- Financial Reports collaboration highlights how the system gathers stored financial data, processes it, and presents meaningful summaries and charts to the user.

Together, these collaboration diagrams provide a complete view of system communication in FinTrac and strengthen the overall design documentation by clearly explaining object-level interactions.

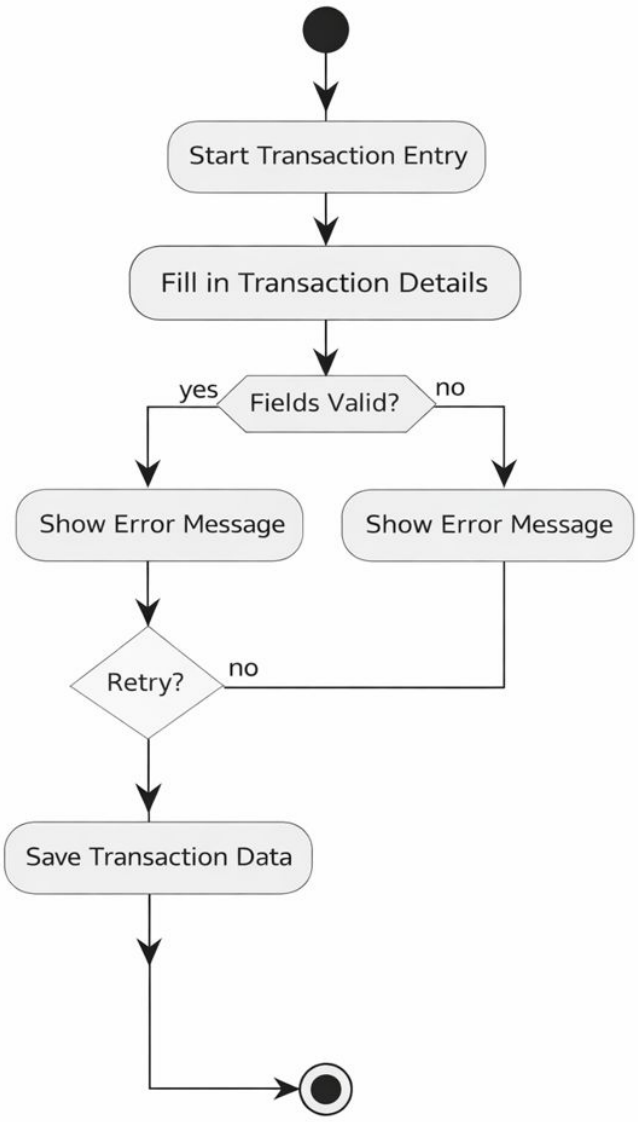
5.3 Activity Diagram

1. An Activity Diagram is a type of UML diagram that represents the flow of activities within a system. It works similar to a flowchart, but it focuses on how a process moves from one step to another, including decision points and possible parallel actions.
2. In the FinTrac project, activity diagrams help in understanding the workflows of key features. Since the application includes multiple modules such as Login, Transaction Management, Budget Planning, Savings Goals, and Financial Reports, separate activity diagrams are used for each module to clearly represent their individual processes and interactions.

Activity Diagram for Transaction Management

Steps in the Process

- User selects the option to add a new transaction.
- System displays the transaction entry form and checks whether all required fields (amount, category, type) are filled.
- If any field is empty or invalid → an error message is shown to the user.
- If all details are valid → the system processes the transaction data.
- System saves the transaction details in the storage layer linked to the user's account.
- Confirmation message is displayed, and the updated transaction is reflected on the dashboard.

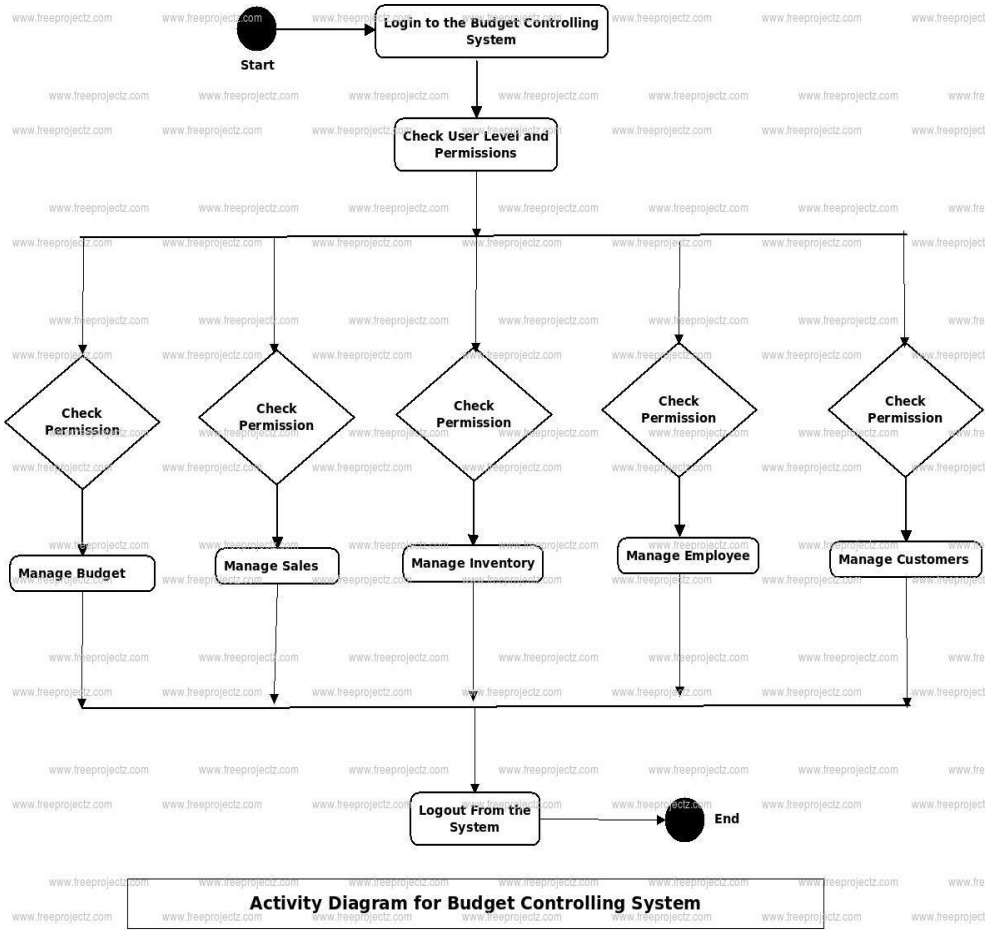

Start Transaction Entry
Fill in Transaction Details
Fields Valid?
yes
no
Show Error Message
Show Error Message
Retry?
no
Save Transaction Data

3. Activity Diagram for Budget Management

Steps in the Process

- User opens the Budget Management section from the dashboard.
- System displays the budget creation or update form.
- User enters budget details such as category and budget limit.
- System checks whether all required fields are filled correctly.
- If any field is missing or invalid → an error message is shown to the user.
- If all details are valid → the system saves the budget information linked to the user account.
- System updates the budget status and spending summary.

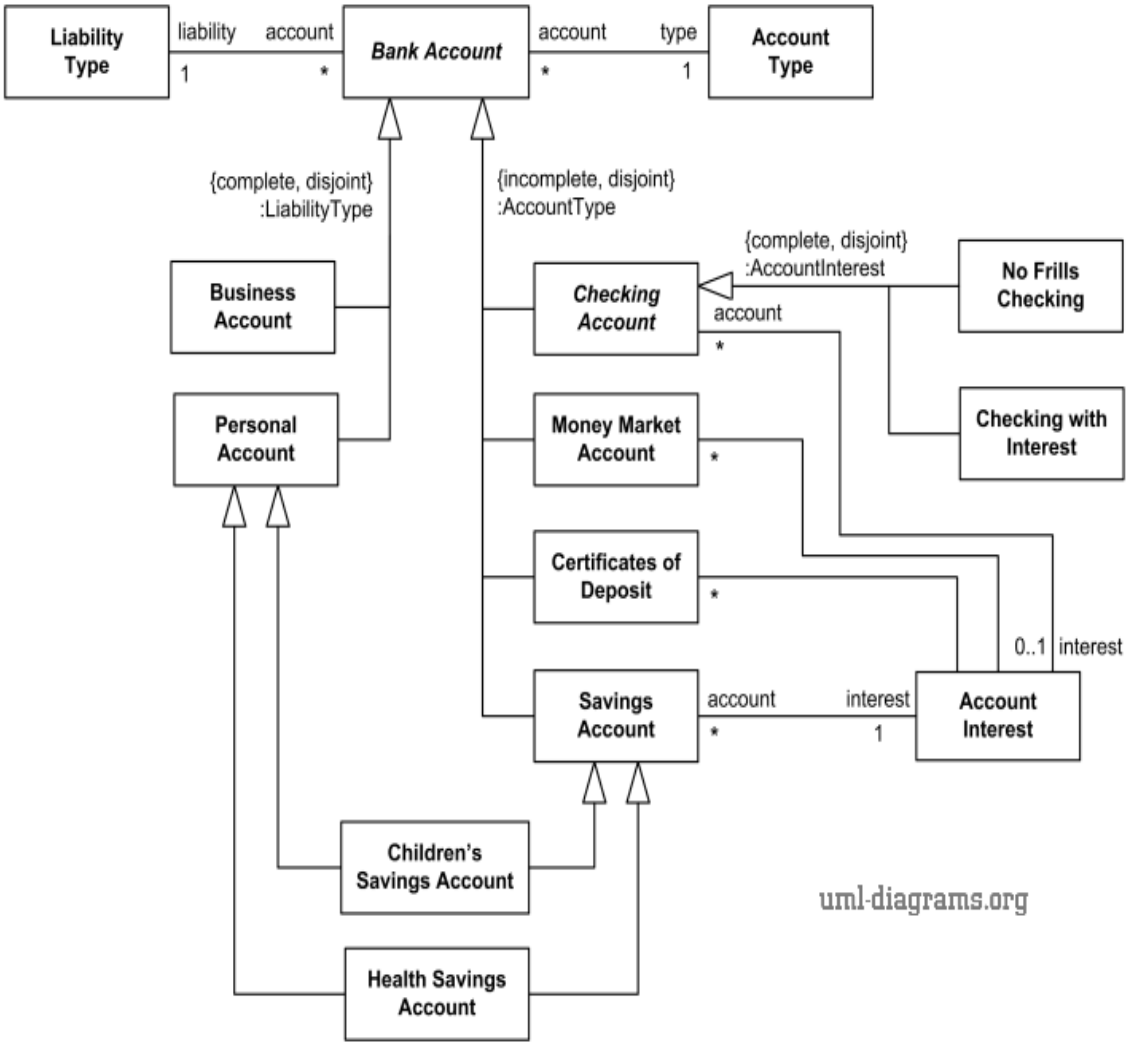
Confirmation message is displayed to the user, and the process ends.



4. Activity Diagram for Saving Goal Management

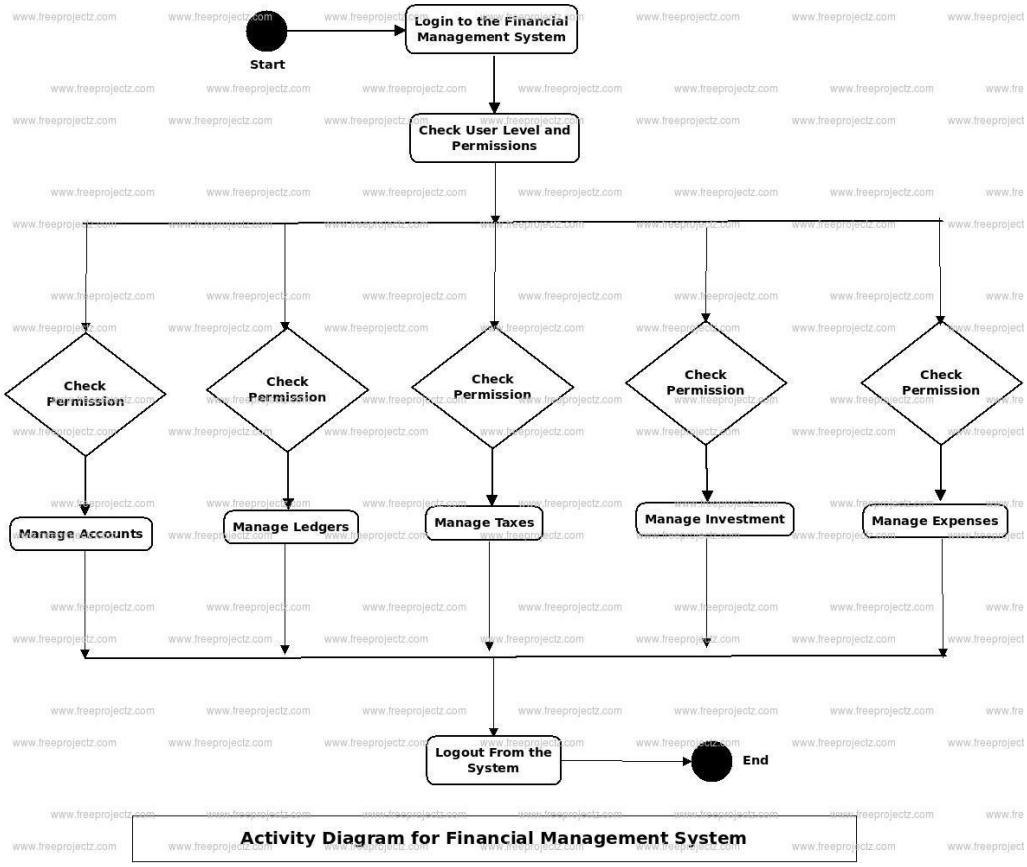
Steps in the Process

1. User opens the Savings Goal section from the dashboard.
2. System displays the savings goal creation form.
3. User enters savings goal details such as goal name, target amount, and target date.
4. System validates the entered details to ensure all required fields are filled correctly.
5. If any field is missing or invalid → an error message is shown to the user.
6. If all details are valid → the system saves the savings goal information linked to the user account.
7. System calculates and updates the goal progress.
8. Confirmation message is displayed to the user, and the process ends.



5. Activity Diagram for Steps in the Process

- User opens the Financial Reports section from the dashboard.
- System retrieves stored transaction, budget, and savings data.
- System processes the data to calculate totals, summaries, and trends.
- User selects filters such as date range or category (if required).
- System updates the report based on the selected filters.
- System generates visual summaries such as charts or tables.
- Financial report is displayed to the user in an easy-to-understand format.
- User reviews the report, and the process ends.



Activity diagrams for FinTrac clearly explain how different workflows operate within the system:

- Transaction Management: Handles proper entry, validation, and storage of income and expense records.
- Budget Management: Supports planning, validating, and saving budget limits for better spending control.
- Savings Goals: Manages goal creation, progress tracking, and completion updates.
- Financial Reports: Retrieves and processes stored data to generate meaningful summaries and insights.

By breaking down these activities into clear steps, the activity diagrams help visualize how the system behaves in different situations. This makes development, testing, debugging, and future enhancements easier and more structured.

5. Database Design

The database acts as the backbone of FinTrac because it stores all user-related financial data such as transactions, budgets, savings goals, and account information. For this project, a structured storage approach is used to manage data efficiently and securely. Instead of relying on traditional table-based storage, the system organizes information in logical data collections linked to individual users.

To clearly represent the database design in the project report, ER diagrams (Entity–Relationship diagrams) or conceptual schemas are used. These diagrams help explain how different entities such as users, transactions, budgets, and goals are related to each other. They provide a clear understanding of data flow and relationships, making the database structure easier to understand and maintain.

Main Entities in FinTrac Database

1. **User Collection**
    - userId (Primary Key)
    - name
    - email

- password
- dateOfSignup

2. **Account Collection**

    - accountId (Primary Key)
    - userId (Foreign Key → User)
    - accountName (Bank, Wallet, Cash, etc.)
    - accountType
    - balance
    - createdAt

3. **Transaction Collection**

    - transactionId (Primary Key)
    - userId (Foreign Key → User)
    - accountId (Foreign Key → Account)
    - type (Income / Expense)
    - category
    - amount
    - description
    - transactionDate

4. **Budget Collection**

    - budgetId (Primary Key)
    - userId (Foreign Key → User)
    - category
    - budgetLimit
    - month
    - year
    - createdAt

5. **Saving Goal Collection**

    - goalId (Primary Key)
    - userId (Foreign Key → User)
    - goalName
    - targetAmount
    - savedAmount
    - targetDate

- createdAt

6. **Report Collection**

    - reportId (Primary Key)
    - userId (Foreign Key → User)
    - reportType (Monthly, Category-wise, Summary)
    - generatedDate

Entity Relationship Summary

- One User can have multiple Accounts, Transactions, Budgets, and Savings Goals.
- Each Transaction is linked to one User and one Account.
- Budgets and Savings Goals are always associated with a single User.
- Reports are generated based on stored financial data and linked to the user.

Relationships

- **User ↔ Accounts → One-to-Many**
  (One user can create and manage multiple financial accounts such as bank, wallet, or cash.)

- **User ↔ Transactions → One-to-Many**
  (One user can record multiple income and expense transactions.)

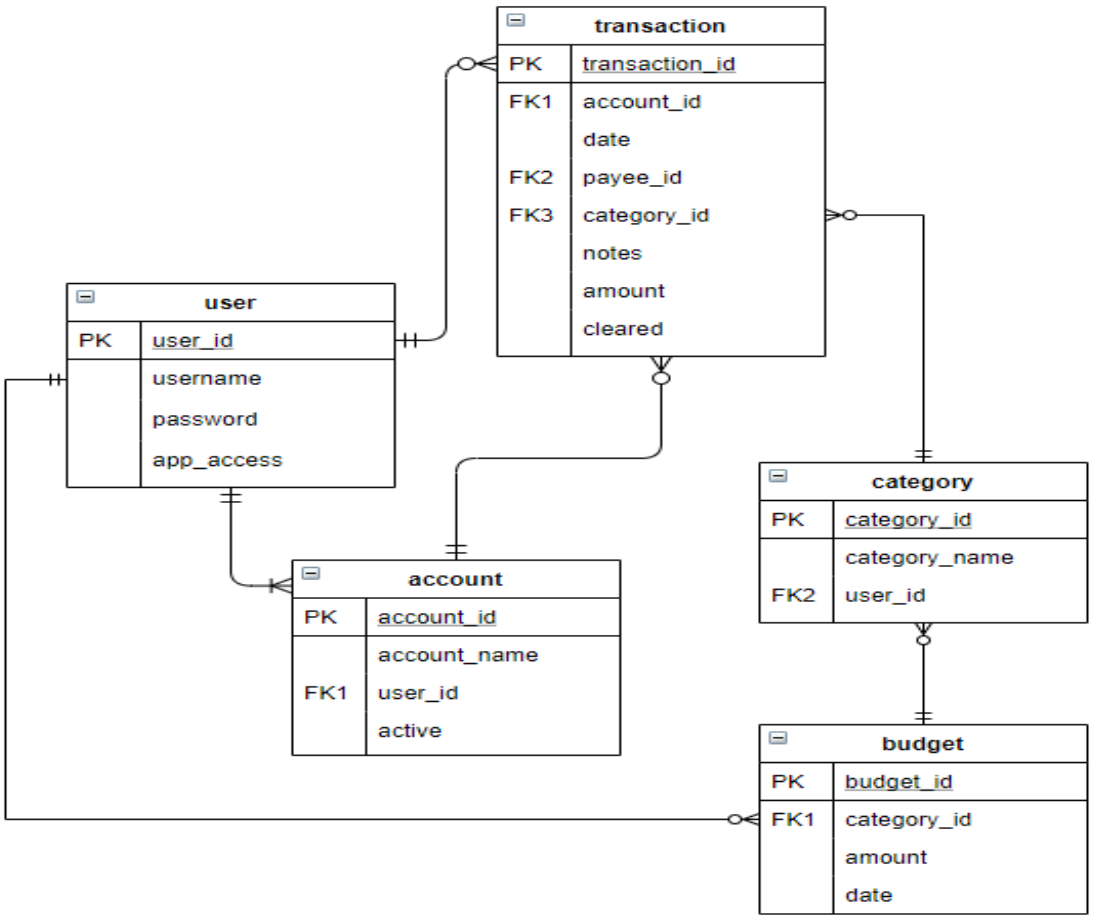
- **User ↔ Budgets → One-to-Many**
  (One user can set multiple budgets for different categories or months.)

- **User ↔ Savings Goals → One-to-Many**
  (One user can create and track multiple savings goals.)

- **Accounts ↔ Transactions → One-to-Many**
  (One account can have multiple transactions linked to it.)

- **User ↔ Financial Reports → Indirect Relationship**
  (Reports are generated using transaction, budget, and savings data related to the user.)

- **Admin (Future Role) →** Will have privileges to manage system-level configurations, monitor usage, and maintain application data without accessing individual user financial details.


transaction
PK transaction_id
FK1 account_id
date
FK2 payee_id
FK3 category_id
notes
amount
cleared
user
PK user_id
username
password
app_access
account
PK account_id
account_name
FK1 user_id
active
category
PK category_id
category_name
FK2 user_id
budget
PK budget_id
FK1 category_id
amount
date

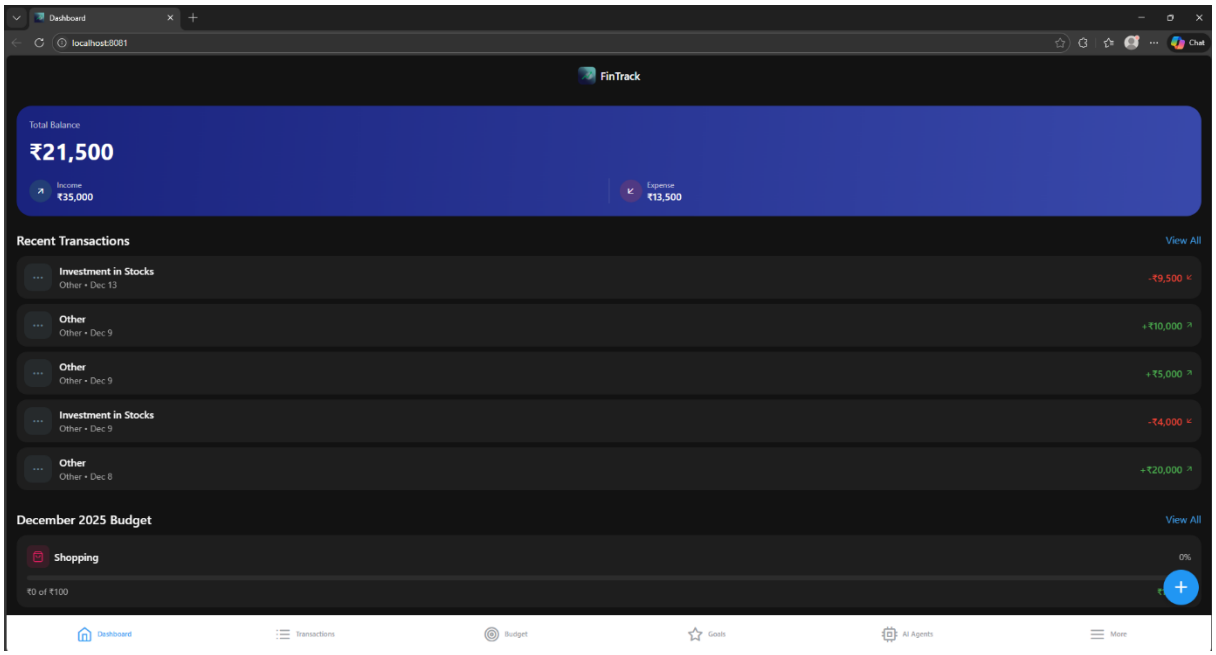
6. Implementation

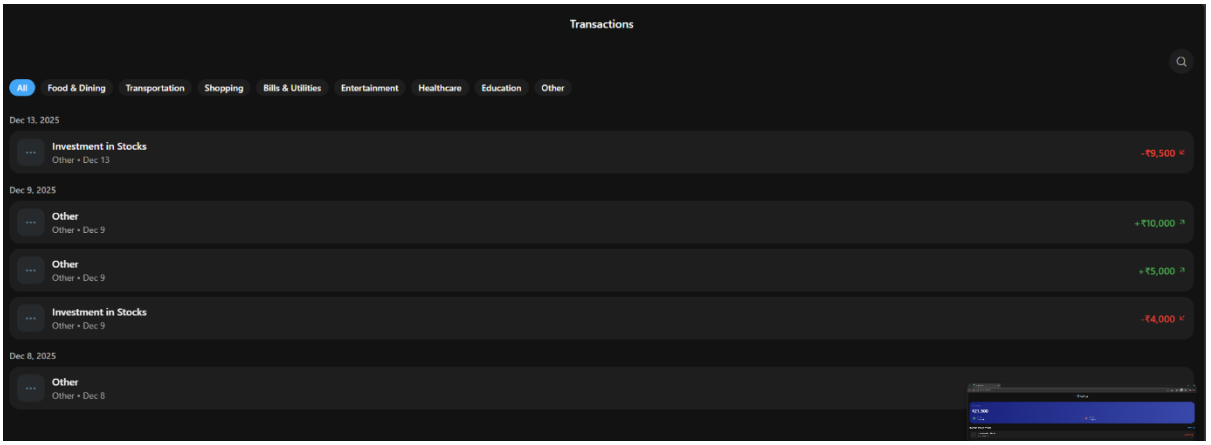
// App.Tsx

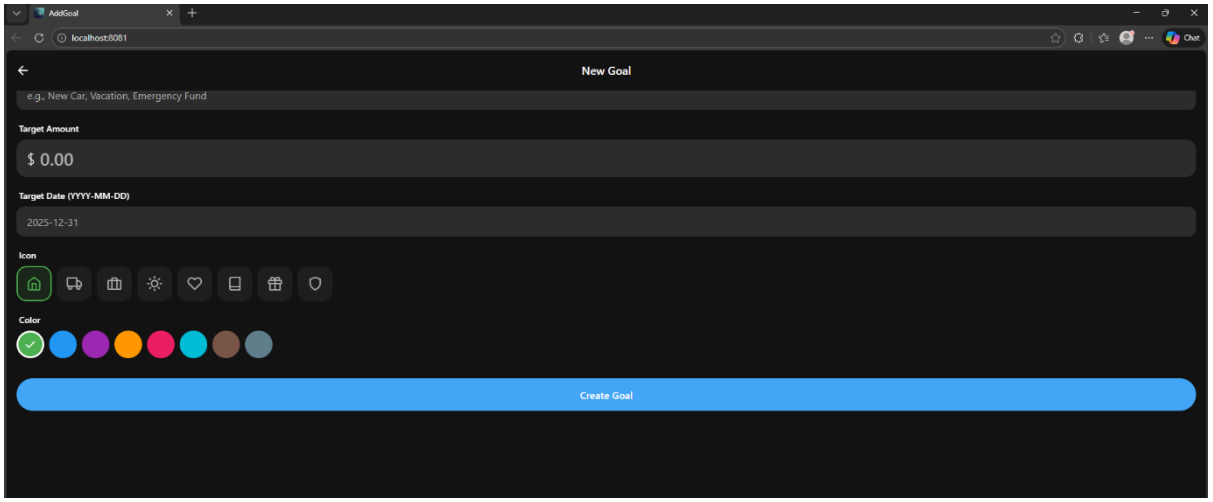
```
import React from "react";
import { StyleSheet } from "react-native";
import { NavigationContainer } from "@react-navigation/native";
import { GestureHandlerRootView } from "react-native-gesture-handler";
import { KeyboardProvider } from "react-native-keyboard-controller";
import { SafeAreaProvider } from "react-native-safe-area-context";
import { StatusBar } from "expo-status-bar";

import { QueryClientProvider } from "@tanstack/react-query";
import { queryClient } from "@/lib/query-client";

import RootStackNavigator from "@/navigation/RootStackNavigator";
import { ErrorBoundary } from "@/components/ErrorBoundary";
import { AuthProvider } from "@/contexts/AuthContext";
import { FinanceProvider } from "@/contexts/FinanceContext";

export default function App() {
  return (
    <ErrorBoundary>
      <QueryClientProvider client={queryClient}>
        <AuthProvider>
          <FinanceProvider>
            <SafeAreaProvider>
              <GestureHandlerRootView style={styles.root}>
                <KeyboardProvider>
                  <NavigationContainer>
                    <RootStackNavigator />
                  </NavigationContainer>
                  <StatusBar style="auto" />
                </KeyboardProvider>
              </GestureHandlerRootView>
            </SafeAreaProvider>
```

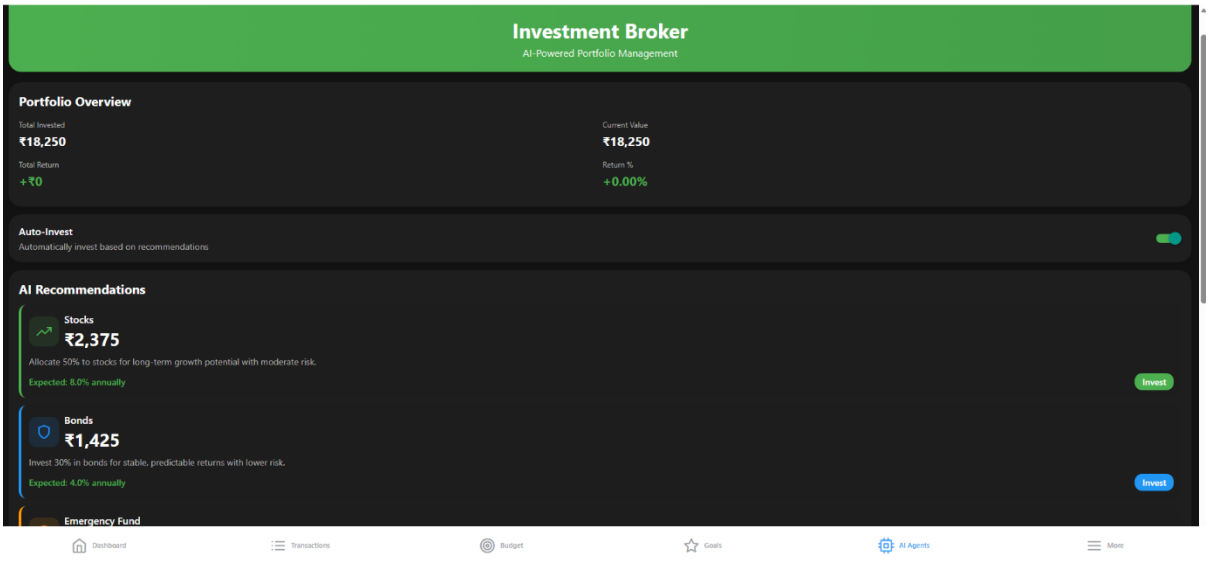
```
        </FinanceProvider>
      </AuthProvider>
    </QueryClientProvider>
  </ErrorBoundary>
 );
}

const styles = StyleSheet.create({
 root: {
  flex: 1,
 },
});
```

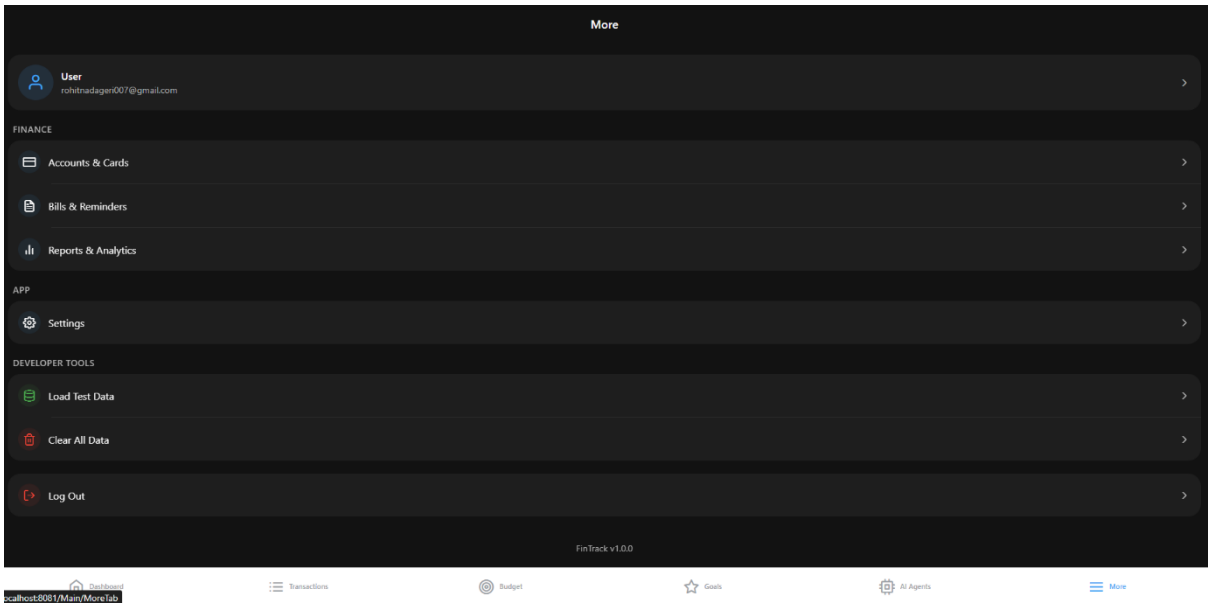
6.1 Screenshots

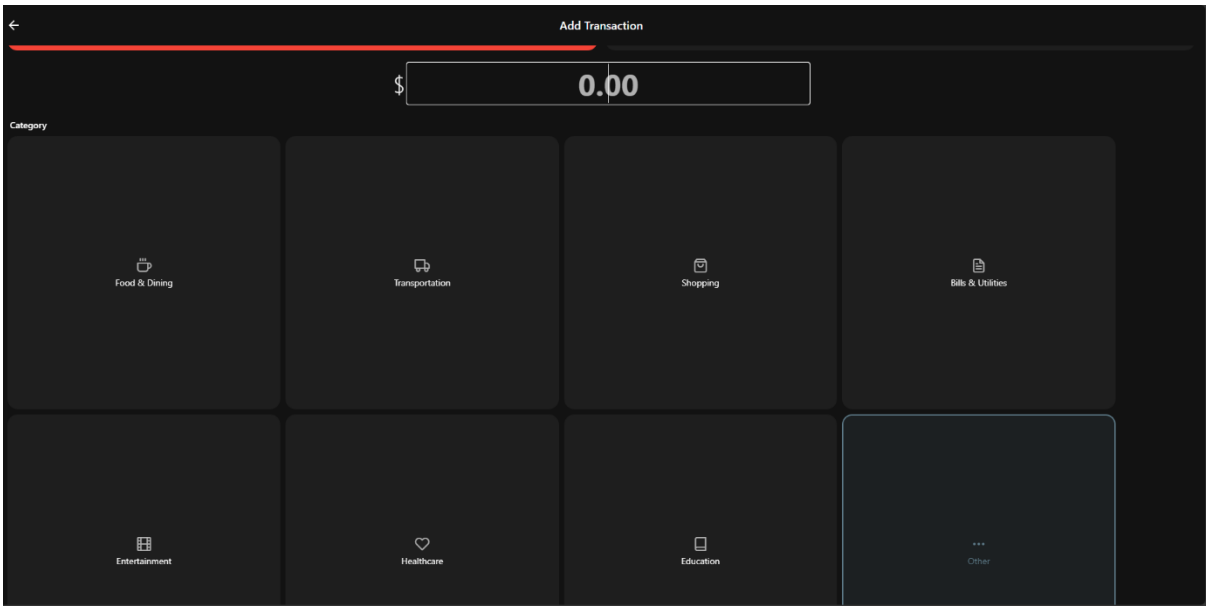





AddGoal
localhost:8081
Chat
New Goal
e.g., New Car, Vacation, Emergency Fund
Target Amount
$ 0.00
Target Date (YYYY-MM-DD)
2025-12-31
Icon
Color
Create Goal


Investment Broker
AI-Powered Portfolio Management
Portfolio Overview
Total Invested
₹18,250
Current Value
₹18,250
Total Return
+₹0
Return %
+0.00%
Auto-Invest
Automatically invest based on recommendations
AI Recommendations
Stocks
₹2,375
Allocate 50% to stocks for long-term growth potential with moderate risk.
Expected: 8.0% annually
Invest
Bonds
₹1,425
Invest 30% in bonds for stable, predictable returns with lower risk.
Expected: 4.0% annually
Invest
Emergency Fund
Dashboard
Transactions
Budget
Goals
AI Agents
More


More
User
rohitnadageri007@gmail.com
FINANCE
Accounts & Cards
Bills & Reminders
Reports & Analytics
APP
Settings
DEVELOPER TOOLS
Load Test Data
Clear All Data
Log Out
FinTrack v1.0.0
Dashboard
Transactions
Budget
Goals
AI Agents
More
localhost:8081/Main/MoreTab


Add Transaction
$
0.00
Category
Food & Dining
Transportation
Shopping
Bills & Utilities
Entertainment
Healthcare
Education
Other

7. Software Testing

7.1 Introduction

Testing is one of the most essential phases of software development. No matter how carefully an application is designed and coded, errors or bugs may still occur. Testing helps in identifying such issues before the software is released to real users, ensuring reliability and correctness.

For the FinTrac project, testing plays a crucial role in verifying that financial features such as Login, Transaction Management, Budget Planning, Savings Goals, and Financial Reports function correctly in real-world usage. Since FinTrac handles sensitive financial data, testing also focuses on data accuracy, security, and system stability.

The main objectives of testing in FinTrac are:

- To verify that the system meets the requirements specified in the Software Requirement Specification (SRS).
- To ensure functional correctness, i.e., each module works as expected.
- To test non-functional aspects such as performance, usability, and reliability.
- To confirm that the system handles both valid and invalid inputs without crashing.

7.2 Testing Strategy

For the FinTrac project, a combination of Black Box and White Box testing techniques was applied.

- Black Box Testing:

This testing focuses on system inputs and outputs without considering internal code structure.

Example: Entering incorrect login credentials and verifying whether the system displays an appropriate error message.

- White Box Testing:

This testing examines the internal logic of the application, including conditions, loops, and data handling.

Example: Verifying whether transaction validation logic and data storage functions work correctly.

Additionally, the following testing levels were performed:

- Unit Testing:

Testing individual modules such as Login, Add Transaction, Budget Creation, and Savings Goal modules.

- Integration Testing:

Testing combined workflows such as “Login → Add Transaction → Update Budget → View Reports”.

- System Testing:

Testing the complete FinTrac application in a real environment to ensure all components work together smoothly.

- User Acceptance Testing (UAT):

Evaluating whether the application is user-friendly and meets the expectations of end users.

7.3 Test Plan

The test plan defines what needs to be tested, how it will be tested, and the environment in which testing is performed. For FinTrac, the test plan is designed as follows:

Test Objectives:

- Ensure all functional modules (Login, Transactions, Budgets, Savings Goals, Reports) work correctly.
- Ensure correct behavior for valid and invalid user inputs.
- Ensure financial data is stored, retrieved, and displayed accurately.

Test Items:

- Login Module
- Transaction Management Module
- Budget Management Module
- Savings Goals Module
- Financial Reports Module

Test Environment:

- Application Type: Web / Mobile Application
- Frontend: React Native / React
- Backend Logic: JavaScript-based application logic
- Storage: Local Storage / Database
- Operating System: Windows 10 (development), Android (testing)
- Browser: Google Chrome, Microsoft Edge

7.3 Test Cases

Below are sample test cases written in tabular format.

Test Case 1: Transaction Management

Test Case ID	Description	Input	Expected Output	Actual Output	Status
TM01	Add valid transaction	Amount, category, type	Transaction saved	Saved successfully	Pass
TM02	Invalid amount	Negative amount	Error message	Error shown	Pass
TM03	Empty fields	Blank input	Warning message	Warning shown	Pass
TM04	Delete transaction	Select transaction	Transaction removed	Deleted successfully	Pass

Test Case 2: Budget Management

Test Case ID	Description	Input	Expected Output	Actual Output	Status
BM01	Create valid budget	Category + limit	Budget saved	Saved successfully	Pass
BM02	Invalid budget limit	Negative limit	Error message	Error shown	Pass
BM03	Empty category	Blank input	Warning message	Warning shown	Pass
BM04	Edit budget	Updated limit	Budget updated	Updated successfully	Pass

Test Case 3: Saving Goal Management

Test Case ID	Description	Input	Expected Output	Actual Output	Status
SG01	Create savings goal	Target + date	Goal created	Created successfully	Pass
SG02	Invalid target	Negative value	Error message	Error shown	Pass
SG03	Empty fields	Blank input	Warning message	Warning shown	Pass
SG04	Update savings goal	Add amount	Progress updated	Updated successfully	Pass

Test Case 4: Financial Report

Test Case ID	Description	Input	Expected Output	Actual Output	Status
FR01	View monthly report	Select month	Report displayed	Displayed successfully	Pass
FR02	Filter by category	Select category	Filtered report	Filtered correctly	Pass
FR03	No data available	Empty dataset	No data message	Message shown	Pass
FR04	Change date range	Custom range	Updated report	Updated correctly	Pass

7.4 Test Results

The results of testing for the FinTrac system showed that:

- All major modules were functioning correctly as per the requirements.
- The system handled both valid and invalid inputs appropriately.
- Core features such as transaction entry, budget creation, savings goal tracking, and report generation worked as expected.
- Data storage and retrieval operations were accurate and consistent.

- No critical or blocking bugs remained unresolved after testing.

Some minor issues observed during testing:

1. Financial reports took slightly longer to load when a large number of transactions were present.
   (Resolved by optimizing data processing logic).

2. Budget update action occasionally required page refresh to reflect changes.
   (Resolved by improving state update handling).

7.5 Conclusion of Testing

The testing phase confirmed that the FinTrac system is stable, reliable, and user-friendly. Most test cases passed successfully, and the minor issues identified during testing were resolved promptly.

Through systematic testing, confidence was gained that the application can be safely used for managing personal finances. Testing also verified that both functional requirements (such as login, transaction management, and data storage) and non-functional requirements (such as performance, usability, and reliability) were met effectively.

8. Conclusion

The FinTrac project was developed with the objective of providing a simple, efficient, and practical personal finance management solution for students and working individuals who want better control over their income, expenses, and savings. Throughout the design and implementation phases, emphasis was placed on building features that are useful in daily financial planning, such as Transaction Management, Budget Planning, Savings Goal Tracking, and Financial Reports.

From the initial requirement analysis to system testing, each module of the application was carefully designed and implemented. The login and authentication mechanism ensures that only registered users can access personal financial data, thereby maintaining privacy and security. At the same time, the system design remains user-friendly and easy to navigate, making it suitable even for non-technical users.

During the testing phase, all major functionalities performed as expected. The system handled both valid and invalid inputs efficiently, and data storage and retrieval operations were accurate. Minor issues identified during testing were resolved promptly, improving overall system stability and performance. The lightweight design of FinTrac allows it to run smoothly without the need for high-end hardware or complex configurations.

In conclusion, the FinTrac project successfully achieved its intended objectives. It not only demonstrates the technical knowledge and development skills gained during the course but also delivers a practical and reliable solution for managing personal finances. The application has strong potential for future enhancements and real-world usage, making it a valuable and meaningful project outcome.

9. Future Enhancements

Though the project is currently running alright, there exist numerous avenues through which to advance with it. A few recommendations to make in the future include:

- **Cloud Synchronization**

Currently, FinTrac stores data locally. In the future, cloud integration can be added so users can access their financial data across multiple devices securely.

- **Bank Integration and Automatic Transaction Import**

FinTrac can be enhanced by integrating with bank APIs to automatically fetch transactions, reducing manual data entry and improving accuracy.

- **Advanced Financial Analytics**

More detailed analytics such as spending predictions, trend analysis, and personalized insights can be introduced to help users make better financial decisions.

- **Reminder and Notification System**

A reminder feature can be added to notify users about upcoming bills, budget limits, or savings goal deadlines through notifications or emails.

- **Multi-Currency Support**

Supporting multiple currencies will make the application useful for users who travel frequently or manage finances across different countries.

- **Export and Report Sharing**

Users can be given the option to export financial reports in formats like PDF or Excel for personal records or sharing with others.

- **Role-Based Access and Admin Panel**

An admin panel can be introduced in the future to manage system settings, user support, and application monitoring without accessing private user data.These changes can enable the platform to be more interactive, smarter and more helpful to many users.

These future enhancements will make FinTrac more powerful, flexible, and suitable for large-scale real-world usage while maintaining ease of use and security.

Appendix A – Bibliography

When I was developing the FinTrac project, I perused various materials to learn concepts, tools and also some design practices. These sources were quite helpful I had to plan the project step by step and construct it.

1 Ian Sommerville, Software Engineering, 10th Edition, Pearson Education, 2016.

2 Pressman, R. S., Software Engineering: A Practitioner's Approach, 8th Edition, McGraw-Hill Education, 2015.

3 React Documentation –
https://react.dev/

4 React Native Documentation –
https://reactnative.dev/

5 JavaScript Documentation (MDN Web Docs) –
https://developer.mozilla.org/en-US/docs/Web/JavaScript

6 UML Diagrams and Modeling –
https://www.uml-diagrams.org/

7 Financial Application Design Concepts –
https://www.geeksforgeeks.org/financial-management/

8 Software Testing Concepts –
https://www.geeksforgeeks.org/software-testing/

9 GitHub Documentation –
https://docs.github.com/

10 Visual Studio Code Documentation –
https://code.visualstudio.com/docs

Appendix B – User Manual

This handbook is prepared to guide both new and existing users of FinTrac. It explains how to register, log in, manage financial data, and handle common issues while using the application..

1. Introduction

FinTrac is a personal finance management application that helps users track income and expenses, create budgets, manage savings goals, and view financial reports. This manual explains how to use each feature in a simple, step-by-step manner.

2. System Requirements

- Operating System: Windows 7 or later, Linux, or macOS
- Browser: Latest version of Chrome / Firefox / Edge
- RAM: Minimum 4 GB recommended
- Internet: Required for login and data synchronization

3. Accessing the Application

- Open your web browser.
- Enter the FinTrac application URL (provided by the developer/admin).
- The home page will load with login and signup options.

4. User Registration and Login

- New User (Sign Up):
  - Click on “Sign Up” from the navigation bar.
  - Enter your name, email, password, and confirm password.
  - Submit the form to create your account.
- Existing User (Login):
  - Click on “Login.”
  - Enter your registered email and password.
  - After successful login, you will be redirected to the Dashboard.

Note: *If wrong credentials are entered, an error message will be shown.*

5. User Dashboard

- After login, the Dashboard appears with the following options:
- Transaction Management
- Budget Management
- Savings Goals
- Financial Reports
- Logout

6. Managing Transactions

Click on Transaction Management from the dashboard.

To add a transaction:

- Enter transaction details such as type, category, amount, and date.
- Click Save.
- A confirmation message will be shown.

To view transactions:

- A list of all saved transactions will be displayed.

7. Budget Management

Click on Budget Management.

- Select a category.
- Enter the budget limit.
- Click Save.

The system tracks expenses against the budget.

8. Savings Goal Management

Click on Savings Goals.

- Enter goal name, target amount, and target date.
- Click Save.
- The system tracks savings progress.

9. Financial Reports

Click on Financial Reports.

Users can view:

- Monthly income and expense summary
- Category-wise analysis
- Budget and savings overview

10. Logging Out

- Click on Logout from the dashboard to end the session securely.

11. Troubleshooting

- Unable to login: Check credentials and internet connection.
- Transaction not saving: Ensure all required fields are filled.
- Reports not loading: Check filters and data availability.

12. Help & Assistance

For support or bug reporting, contact the developer/admin using the support email provided on the application homepage.